

EVALUATION OF PINNS FOR SOLVING PDES

A PROJECT REPORT

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE BACHELOR OF SCIENCE IN COMPUTATIONAL
MATHEMATICS

BY

SHREESH BHATTARAI

4th YEAR / 2nd SEMESTER



DEPARTMENT OF MATHEMATICS
SCHOOL OF SCIENCE
KATHMANDU UNIVERSITY
DHULIKHEL, NEPAL

AUGUST, 2025

DEDICATION

To

my parents

Dr. Anoj Bhattarai and Mrs. Anu Bhattarai

& my brother

Mr. Subrat Bhattarai

DECLARATION

I, Shreesh Bhattarai, hereby declare that the work contained herein is entirely my own, except where states otherwise by reference or acknowledgment, and has not been published or submitted elsewhere, in whole or in part, for the requirement for any other degree or professional qualification. Any literature, data or works done by others and cited within this report has been given due acknowledgment and listed in the reference section.

Shreesh Bhattarai

KU Registration number: 028301-20

Date:

CERTIFICATION

This report entitled "Evaluation of PINNs for solving PDEs" is carried out under my supervision for the specified entire period satisfactorily, and is hereby certified as original work done by Shreesh Bhattarai in partial fulfillment of the requirements for the Bachelor's of Science degree in Computational Mathematics, Kathmandu University, Dhulikhel, Nepal.

Dr. Harish Chandra Bhandari

Department of Mathematics,
School of Science
Kathmandu University
Dhulikhel, Kavre, Nepal
Date:

APPROVED BY:

I hereby declare that the candidate qualifies to submit this work as Bachelor's of Science report to the Examination Committee.

Prof. Dr. Rabindra Kayastha

Head of the Department
Department of Mathematics
School of Science
Kathmandu University
Dhulikhel, Kavre, Nepal
Date:

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to Dr. Harish Chandra Bhandari, whose supervision and guidance were instrumental in the execution and successful completion of this project. I am deeply appreciative of the valuable suggestions and continuous support provided throughout the process.

I am also thankful to the Department of Mathematics for their encouragement and assistance, which played a significant role in facilitating the progress and completion of this work.

Finally, I extend my heartfelt thanks to everyone who supported me during this journey. Their collaboration, encouragement, and provision of necessary resources greatly enhanced the overall quality and success of my project.

ABSTRACT

The heat equation, a fundamental partial differential equation (PDE), plays a central role in modeling heat transfer across a wide range of engineering applications. Traditional Finite Difference Methods (FDMs), including Explicit, Implicit, and Crank-Nicolson schemes, rely on structured grids but struggle with complex geometries and stability constraints. In contrast, Physics-Informed Neural Networks (PINNs) present a mesh-free alternative by embedding PDE constraints directly into the loss function of neural networks. This study conducts a comparative evaluation of PINNs and FDMs for solving 1D, 2D, and 3D heat equations under coarse spatial discretization, assessing their accuracy, computational efficiency, and sensitivity to hyperparameter configurations. By addressing the lack of comprehensive benchmarking across dimensionalities and evaluating classical method against modern approach, this work aims to elucidate the practical strengths and limitations of PINNs and establish a foundation for their application to complex, real-world PDE systems.

Keywords: Physics-Informed Neural Network, Heat Equation, Partial Differential Equation, Finite Difference Method

CONTENTS

DEDICATION	I
DECLARATION	II
CERTIFICATION	III
ACKNOWLEDGEMENTS	IV
ABSTRACT	V
LIST OF FIGURES	VIII
LIST OF TABLES	IX
LIST OF ACRONYMS	X
LIST OF SYMBOLS	XI
1 Introduction	1
1.1 Introduction	1
1.2 Objective	1
1.3 Problem Statement	2
1.4 Heat Equations	2
1.4.1 One-Dimensional (1D) Heat Equation	2
1.4.2 Two-Dimensional (2D) Heat Equation	2
1.4.3 Three-Dimensional (3D) Heat Equation	3
1.5 Heat Diffusion from a Single Pulse	3
1.6 Thermal Diffusivity	3
1.7 Analytical Solution	4
1.8 Numerical Method	4
1.9 Physics-Informed Neural Networks (PINNs)	4
2 Literature Review	5
2.1 Introduction	5
2.2 Background	5

2.2.1	Theory of Finite Difference Methods	5
2.2.2	Theory of PINNs	6
2.3	Related Works	6
2.4	Research Gaps	11
3	Mathematical Framework	12
3.1	Analytical Solutions	12
3.1.1	Analytical Solution of 1D Heat Equation	12
3.1.2	Analytical Solution of 2D Heat Equation	12
3.1.3	Analytical Solution of 3D Heat Equation	13
3.2	Finite Difference Method	13
3.2.1	Explicit Scheme	13
3.2.2	Implicit Scheme	14
3.2.3	Crank-Nicolson Scheme	15
3.3	PINN Working Mechanism	16
4	Methodology	19
4.1	Computational Environment	19
4.2	Simulation Setup	19
4.3	Analytical Solutions	22
4.4	Implementation of FDM Schemes	22
4.4.1	Explicit Scheme	22
4.4.2	Implicit Scheme	23
4.4.3	Crank-Nicolson Scheme	23
4.5	Implementation of PINN	24
4.5.1	Latin Hypercube Sampling	25
4.5.2	Training	26
4.6	Performance Evaluation	27
5	Results & Discussion	29
5.1	Optimal Configurations	29
5.2	Performance Comparison	30
5.3	Accuracy Analysis	35
5.4	Discussion	36
5.4.1	Loss Convergence	36
5.4.2	Benchmarks	39
5.5	Limitations	40
6	Conclusion	42
7	Future Works	43

LIST OF FIGURES

3.1	Flowchart representing PINN's working mechanism	18
4.1	Distribution of collocation points for 1D heat equation in a rod.	28
4.2	Distribution of collocation points for 2D heat equation in a sheet.	28
4.3	Distribution of collocation points for 3D heat equation in a cube.	28
5.1	1D heat equation: solution and error comparison.	32
5.2	2D heat equation: solution and error comparison.	33
5.3	3D heat equation: solution and error comparison.	34
5.4	Loss convergence of 1D heat equation by PINN	37
5.5	Loss convergence of 2D heat equation by PINN	38
5.6	Loss convergence of 3D heat equation by PINN	38
5.7	Execution time comparison across dimensions for each methods	40
5.8	FDM vs PINN discretization cost	40
5.9	Error metrics (L_2 , L_∞ , Relative L_2) by dimension	41

LIST OF TABLES

2.1	Summary of key contributions of related works	10
4.1	Hardware specifications for kaggle notebook environment	19
4.2	Network architecture and hyperparameters	24
4.3	Collocation points distribution by dimension	25
4.4	Loss weighting strategies for PINN training	26
5.1	Optimal PINN configurations by dimension	29
5.2	Comprehensive performance metrics	30
5.3	Accuracy comparison (%)	35
5.4	Average computational performance comparison across all dimensions .	39
5.5	Spatiotemporal discretization comparison	39

LIST OF ACRONYMS

PINNs	– Physics-Informed Neural Networks
FDM	– Finite Difference Method
LHS	– Latin Hypercube Sampling
PDE	– Partial Differential Equation

LIST OF SYMBOLS

D	Dimension.
W	Watt.
K	Kelvin.
J	Joule.
kg	kilogram.
$u(x, t)$	Temperature function in 1D heat equation, dependent on spatial coordinate x and time t .
$u(x, y, t)$	Temperature function in 2D heat equation, dependent on spatial coordinates (x, y) and time t .
$u(x, y, z, t)$	Temperature function in 3D heat equation, dependent on spatial coordinates (x, y, z) and time t .
x, y, z	Spatial coordinates in 1D, 2D, and 3D domains, respectively, in meter (m).
t	Time variable, in second (s).
α	Thermal diffusivity, ratio of thermal conductivity to heat storage, in $\text{m}^2 \text{s}^{-1}$.
k	Thermal conductivity, in $\text{W m}^{-1} \text{K}^{-1}$.
ρ	Density, in kg m^{-3} .
c_p	Specific heat capacity, in $\text{J kg}^{-1} \text{K}^{-1}$.
$u_0(x), u_0(x, y), u_0(x, y, z)$	Initial temperature profiles at $t = 0$ for 1D, 2D, and 3D heat equations.
L	Length of the domain in 1D, 2D, or 3D (e.g., rod length or side of square/cube), in meter (m).
b_n	Fourier coefficients for 1D analytical solution, computed from initial condition.
A_{mn}	Double Fourier coefficients for 2D analytical solution, computed from initial condition.
A_{mnp}	Triple Fourier coefficients for 3D analytical solution, computed from initial condition.

λ_{mn}	Eigenvalue for 2D heat equation, defined as $\lambda_{mn}^2 = \pi^2 \left(\frac{m^2}{L^2} + \frac{n^2}{L^2} \right)$.
λ_{mnp}	Eigenvalue for 3D heat equation, defined as $\lambda_{mnp}^2 = \pi^2 \left(\frac{m^2}{L^2} + \frac{n^2}{L^2} + \frac{p^2}{L^2} \right)$.
u_i^n	Numerical solution at spatial grid point $x_i = i\Delta x$ and time step $t_n = n\Delta t$.
u_{i+1}^n	Numerical solution at spatial grid point $x_{i+1} = (i+1)\Delta x$ and time step t_n .
u_{i-1}^{n+1}	Numerical solution at spatial grid point $x_{i-1} = (i-1)\Delta x$ and time step $t_{n+1} = (n+1)\Delta t$.
Δx	Spatial grid spacing, in meter (m).
Δt	Time step size, in seconds (s).
r	Dimensionless stability parameter, defined as $r = \frac{\alpha \Delta t}{(\Delta x)^2}$.
λ_{PDE}	Weight parameter of PDE.
λ_{BC}	Weight parameter of BC.
λ_{IC}	Weight parameter of IC.
$\mathcal{L}$	Total loss function in PINNs, combining PDE, boundary, and initial condition losses.
$\mathcal{L}_{PDE}$	PDE residual loss, enforcing the heat equation in PINNs.
$\mathcal{L}_{BC}$	Boundary condition loss, enforcing Dirichlet conditions in PINNs.
$\mathcal{L}_{IC}$	Initial condition loss, enforcing initial temperature in PINNs.
N_p	Number of collocation points for PDE residual in PINNs.
N_b	Number of boundary points for boundary condition loss in PINNs.
N_i	Number of initial points for initial condition loss in PINNs.

CHAPTER 1

Introduction

1.1 Introduction

The heat equation, a fundamental partial differential equation (PDE), models heat transfer in various physical systems, requiring robust numerical methods for near accurate solutions [1, 5]. Traditional Finite Difference Methods (FDMs), including Explicit, Implicit, and Crank-Nicolson schemes, rely on structured grids and face challenges with complex geometries and stability constraints, such as the Courant-Friedrichs-Lewy condition [2, 13, 34]. Physics-Informed Neural Networks (PINNs) offer a mesh-free alternative by embedding PDE governing laws into neural network loss functions, promising flexibility for complex and high-dimensional problems [3, 4]. PINNs use automatic differentiation to compute the exact derivatives for PDE residuals, avoiding numerical discretization errors inherent in traditional methods like FDMs [3, 15]. This project systematically compares PINNs with FDMs for solving 1D, 2D, and 3D heat equations, focusing on accuracy, computational efficiency, and robustness under coarse discretization. By evaluating these methods, the study aims to clarify PINN's practical advantages, address its limitations, and explore multi-dimensional approaches.

1.2 Objective

The goals of this project are:

- To explore the foundation of PINNs, including their neural network architecture and loss function formulation.
- To conduct a comparative study of FDMs and PINNs, fostering a deeper understanding of both mathematical and AI-driven methodologies for solving the heat equation.
- To lay the foundation for studying the project to multi-dimensional or real-world PDE systems, enhancing applicability to practical engineering problems.

1.3 Problem Statement

Solving the heat equation in 1D, 2D, and 3D domains is critical for applications in engineering [5, 23]. Yet, traditional FDMs face limitations in handling higher dimensions and typically require fine grids for accuracy, which significantly increases computational cost and time [2, 13]. PINNs, while flexible and mesh-free, exhibit challenges in scalability, hyperparameter sensitivity, and computational efficiency, for both lower-dimensional and higher-dimensional problems [21, 36, 38]. To the best of our knowledge, no prior study benchmarks PINNs against Explicit, Implicit, and Crank-Nicolson FDMs across these dimensions with coarse discretization, focusing on accuracy and runtime. This project addresses this gap by evaluating both methods to determine their practical efficacy.

1.4 Heat Equations

The following description of heat equations are stated referring to the works of Lawrence Craig Evans [1] and Horatio Scott Carslaw, et al. [5].

1.4.1 One-Dimensional (1D) Heat Equation

The one-dimensional (1D) heat equation describes temperature distribution along a homogeneous rod of length L . It is given by:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}, \quad 0 < x < L, \quad t > 0, \quad (1.1)$$

where x is the spatial coordinate, $u(x, t)$ is the temperature, and α is the thermal diffusivity. Dirichlet boundary conditions fix the temperature at zero:

$$u(0, t) = u(L, t) = 0, \quad t > 0, \quad (1.2)$$

with initial condition:

$$u(x, 0) = u_0(x), \quad 0 < x < L. \quad (1.3)$$

1.4.2 Two-Dimensional (2D) Heat Equation

The two-dimensional (2D) heat equation governs temperature on a rectangular plate of area $[0, L] \times [0, L]$:

$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right), \quad (x, y) \in [0, L]^2, \quad t > 0, \quad (1.4)$$

where (x, y) are spatial coordinates, $u(x, y, t)$ is the temperature, and α is the thermal diffusivity. Dirichlet boundary conditions are:

$$u(x, y, t) = 0, \quad (x, y) \in \partial[0, L]^2, \quad t > 0, \quad (1.5)$$

with initial condition:

$$u(x, y, 0) = u_0(x, y), \quad (x, y) \in [0, L]^2. \quad (1.6)$$

1.4.3 Three-Dimensional (3D) Heat Equation

The three-dimensional (3D) heat equation governs temperature distribution in a cubic domain of volume $[0, L]^3$:

$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right), \quad (x, y, z) \in [0, L]^3, \quad t > 0, \quad (1.7)$$

where (x, y, z) are spatial coordinates, $u(x, y, z, t)$ is the temperature, and α is the thermal diffusivity. Dirichlet boundary conditions are:

$$u(x, y, z, t) = 0, \quad (x, y, z) \in \partial[0, L]^3, \quad t > 0, \quad (1.8)$$

with initial condition:

$$u(x, y, z, 0) = u_0(x, y, z), \quad (x, y, z) \in [0, L]^3. \quad (1.9)$$

1.5 Heat Diffusion from a Single Pulse

Heat diffusion from a single pulse represents a fundamental transient heat transfer problem. An instantaneous heat pulse introduced in a medium causes the temperature to evolve rapidly initially, then gradually, tending toward thermal equilibrium [5, 9]. The impulse response illustrates the diffusive nature of heat transport, where sharp initial profiles broaden over time, quickly losing memory of the initial shape [10].

1.6 Thermal Diffusivity

Thermal diffusivity (α) quantifies the rate of temperature propagation in a material, defined as [9, 11]:

$$\alpha = \frac{k}{\rho c_p}, \quad (1.10)$$

where k is the thermal conductivity ($\text{W m}^{-1} \text{K}^{-1}$), ρ is the density (kg m^{-3}), and c_p is the specific heat capacity ($\text{J kg}^{-1} \text{K}^{-1}$). The SI unit of thermal diffusivity is $\text{m}^2 \text{s}^{-1}$.

[12]. It represents the ratio of a material’s ability to conduct heat to its ability to store heat, determining the speed of thermal equilibrium [11].

1.7 Analytical Solution

An analytical solution, often called an exact or closed-form solution, expresses the solution of a differential equation, such as the 1D, 2D, and 3D heat equations, using standard mathematical functions without numerical approximation [1].

1.8 Numerical Method

A numerical method approximates solutions to differential equations using computational algorithms [2]. A widely used approach is the FDM [8]. FDM approximates solutions to PDEs by discretizing derivatives using differences between function values at discrete grid points [2, 8, 13].

1.9 Physics-Informed Neural Networks (PINNs)

PINNs are deep learning frameworks that incorporate physical laws, described by PDEs, as constraints in the training process [3]. PINNs minimize a loss function combining data fitting and PDE residual terms:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \mathcal{L}_{\text{PDE}}, \quad (1.11)$$

using automatic differentiation to compute derivatives, eliminating mesh generation [3, 15]. PINNs handle complex geometries, sparse data, and both forward and inverse problems [3, 4].

CHAPTER 2

Literature Review

2.1 Introduction

PINNs embed PDE governing laws into neural network loss functions, offering a mesh-free alternative to traditional numerical methods like FDMs [3, 4]. In heat transfer modeling, PINNs bypass grid-generation and stability constraints (here., Courant-Friedrichs-Lewy condition) of FDMs, promising flexibility for complex geometries and dimensions [3, 19, 30]. This study systematically evaluates PINNs against three FDM schemes—Explicit, Implicit, and Crank-Nicolson—for solving 1D, 2D, and 3D heat equations, focusing on accuracy, computational efficiency, and resource usage under coarse discretization.

2.2 Background

As per the prior mentions, the project implements FDM and PINN as the pillars of comparison for solving 1D, 2D, and 3D heat equations. The following subsections detail the theoretical foundations of these methods, highlighting their application to heat transfer problems across multiple dimensions.

2.2.1 Theory of Finite Difference Methods

FDMs discretize PDEs using differences on structured grids. For the heat equation, spatial $(\Delta x, \Delta y, \Delta z)$ and temporal (Δt) domains are divided into uniform grids. The following mathematical descriptions are stated referring to the works of Randall J. LeVeque [2], G.D. Smith [13], Gerald Recktenwald [16], Keith William Morton, et al. [33], Richard Courant, et al. [34].

- **Explicit Scheme:** Uses forward differencing in time and central differencing in space, conditionally stable with CFL condition $\frac{\alpha \Delta t}{(\Delta x)^2} \leq \frac{1}{2}$ (1D), extended to 2D $(\frac{\alpha \Delta t}{(\Delta x)^2} + \frac{\alpha \Delta t}{(\Delta y)^2} \leq \frac{1}{2})$ and 3D $(\frac{\alpha \Delta t}{(\Delta x)^2} + \frac{\alpha \Delta t}{(\Delta y)^2} + \frac{\alpha \Delta t}{(\Delta z)^2} \leq \frac{1}{2})$.

- **Implicit Scheme:** Employs backward differencing, forming tridiagonal (1D) or sparse (2D, 3D) systems, unconditionally stable, solved via Thomas algorithm or sparse solvers.
- **Crank-Nicolson Scheme:** Averages explicit and implicit schemes, second-order accurate, unconditionally stable, solved using sparse solvers as well.

2.2.2 Theory of PINNs

PINNs approximate the solution $u(x, t)$ using a feedforward neural network, minimizing the loss function [3, 4, 19]:

$$\mathcal{L} = \lambda_{\text{PDE}}\mathcal{L}_{\text{PDE}} + \lambda_{\text{BC}}\mathcal{L}_{\text{BC}} + \lambda_{\text{IC}}\mathcal{L}_{\text{IC}}, \quad (2.1)$$

where, λ is the respective weight parameter, and the collocation points are sampled via LHS [14]. Moreover, automatic differentiation computes exact derivatives, avoiding discretization errors [15]. PINNs incorporate physical laws described by PDEs into the training process for loss convergence [3].

2.3 Related Works

FDMs have a long history of successful application to the heat equation. For instance, Strikwerda (2002) lays out foundational time-stepping strategies and stability analysis for 1D heat conduction problems [48]. Similarly, Gerald (2004) presented a comparison of truncation errors for various FDM schemes, specifically Forward Time Central Space (FTCS), Backward Time Central Space (BTCS), and Crank-Nicolson, as a function of time step (Δt) for fixed spatial step (Δx), and vice versa with practical guidance and MATLAB implementations for 1D and 2D heat equations [16]. LeVeque, Randall J. (2007) presented a book on FDMs for Ordinary and Partial Differential Equations, focusing on steady-state and time-dependent problems. This work emphasizes building an understanding of the essential ideas underlying the development, analysis, and practical use of FDM schemes, with stability theory playing a large role [2]. Furthermore, Mebrate (2015) focuses specifically on numerical solutions of the 1D heat equation subject to Dirichlet boundary conditions, demonstrating the accuracy and robustness of FDM on such problems [17]. More recently, Omar, et al. (2023) presented a study that uses and derives the Crank-Nicolson numerical method to solve the heat-diffusion problem, comparing its accuracy, stability, and computational time against the Alternating Directions Implicit method, concluding that Crank-Nicolson is more accurate in initial steps, more stable, and takes less total time to reach final results than ADI [18].

These studies collectively establish FDMs as robust, accurate, and computationally efficient methods for solving heat equations on structured grids, providing a solid foun-

dition for comparisons with mesh-free approaches such as PINNs.

In parallel, Glorot, et al. (2010) presented an investigation into the difficulties of training deep feedforward neural networks using standard gradient descent from random initialization, offering insights into the impact of non-linear activation functions and proposing a new initialization scheme for substantially faster convergence [35]. Then, the foundational work of Raissi et al. (2019) introduced PINNs, validating them on Burgers' and Schrödinger equations, demonstrating mesh-free PDE solutions [3]. Continuing on this, Jagtap, et al. (2019) introduced an adaptive activation function with a scalable hyper-parameter for deep neural networks and PINNs to accelerate convergence and improve accuracy when approximating functions and solving linear and nonlinear PDEs in both forward and inverse problems [7]. Lu Lu, et al. (2020) introduced DeepXDE, a Python library designed for solving differential equations using PINNs, which embed PDEs into the neural network's loss function through automatic differentiation, and introduces a residual-based adaptive refinement (RAR) method to improve PINN training efficiency for both forward and inverse problems [6].

Karniadakis, et al. (2021) presented a comprehensive review of Physics-Informed Machine Learning (PIML), particularly PINNs, demonstrating their seamless integration of observational data and mathematical models to effectively address challenging forward and inverse problems in scientific domains, including those with limited data, high dimensionality, or hidden physics [4]. Then, Wang, et al. (2021) analyzed the fundamental mode of failure in PINNs caused by numerical stiffness leading to unbalanced back-propagated gradients during training, and proposes an adaptive learning rate annealing algorithm and a novel neural network architecture to mitigate these "gradient flow pathologies" and improve PINN predictive accuracy by 50-100 times [37]. Additionally, Cai, et al. (2021) introduced applications of PINNs for various prototype and industrial heat transfer problems, demonstrating their effectiveness in solving ill-posed inverse problems, and highlighting their capability to bridge the gap between computational and experimental heat transfer [24]. Krishnapriyan, et al. (2021) presented a characterization of failure modes in PINNs, demonstrating that these issues stem from optimization difficulties caused by PDE-based soft constraints and complex loss landscapes rather than network expressivity, and proposes curriculum regularization and sequence-to-sequence learning as effective strategies to achieve significantly lower errors [32].

Aygun, et al. (2022) presented PINNs to solve 2D incompressible thermal convection problems, and demonstrating their accuracy and sensitivity to loss term weighting across various channel flow, heated cavity, and blocked channel applications [43]. Xiang, et al. (2022) introduced the hybrid finite difference with the PINN (HFD-PINN),

a novel approach that combines FDM in regular domains with automatic differentiation at complex boundaries within the PINN framework to solve PDEs more efficiently and accurately [42].

Additionally, Grossman, et al. (2023) presented a systematic computational comparison of PINNs and the Finite Element Method (FEM) for numerically solving various linear and nonlinear PDEs, concluding that PINNs generally did not outperform FEM in solution time and accuracy but demonstrated faster evaluation and potential for high-dimensional problems [45]. Xing, Z. et al. (2023) introduced a deep learning method based on PINNs specifically designed for 3D anisotropic steady-state heat conduction problems [27]. Savović, et al. (2023) presented a comparative study of the Explicit FDM and PINN for solving the Burgers' equation across three test problems with various initial and Dirichlet boundary conditions, demonstrating that while both approaches yield very good agreement with analytical solutions, the Explicit FDM generally achieves higher accuracy with sufficiently fine step lengths, though both methods are considered competitive and recommended [41]. Bowman, et al. (2023) presented a study demonstrating that PINNs can accurately surrogate numerical solvers for the 1D heat equation with a source term, specifically for modeling laser-tissue interactions under various boundary conditions, and identifies optimal activation function combinations and the effectiveness of a scaling and translation (S&T) method for improved performance [39]. Zhang, et al. (2023) introduced a novel approach to significantly accelerate the training of PINNs for high-dimensional second-order PDEs by parameterizing the solution as a Gaussian smoothed model and leveraging Stein's Identity to efficiently calculate derivatives without stacked back-propagation, leading to competitive error rates with substantially faster training times [22].

Urbán, et al. (2024) presented a study exploring the optimization process of PINNs, demonstrating that modifications to BFGS-type optimization algorithms and the mean squared error loss function significantly enhance accuracy by orders of magnitude and reduce computational costs, enabling compact networks to achieve precision comparable to traditional numerical methods across diverse physical problems [36]. Moreover, Fabiani, et al. (2024) presented a rigorous stability analysis of PINNs with random projections for solving stiff linear parabolic PDE, providing the first formal proof that properly designed PINNs are consistent and asymptotically stable, and demonstrating their superior numerical approximation accuracy and comparable computational cost relative to traditional implicit schemes [30]. Similarly extending the research, Raissi, et al. (2024) presented a comprehensive review of PINNs, detailing their fundamental principles as a paradigm shift in scientific machine learning for seamlessly integrating data and mathematical models, exploring various practical extensions, and providing a specific example of their utility in data-driven discovery of governing differential equa-

tions for nonlinear dynamical systems [29]. Then, Farmer, et al. (2024) presented an empirical study on optimizing loss component weights in PINNs for modeling laser bio-effects on human skin, proposing a heuristic magnitude normalization method that significantly accelerates convergence, and reduces uncertainty [28]. Likewise, Bhatnagar, et al. (2024) presented an investigation into effectively training PINNs for complex 3D highly nonlinear flow-thermal problems, demonstrating their capability to solve Navier-Stokes equations at high Reynolds numbers and generate superior surrogate models for design optimization by leveraging very sparse solution data and advanced training techniques like hybrid data-physics training, modified learning rate annealing, and Fourier feature embeddings [26]. Aryal (2024) presented a comprehensive study on simulating 2D heat conduction using PINNs, demonstrating their accuracy and adaptability across various heat sources and boundary conditions (including Neumann and Dirichlet), proving them to be a robust and efficient alternative to traditional numerical methods while also highlighting their capabilities and limitations [25]. Zobeiry, et al. (2024) presented PINN for solving conductive heat transfer PDEs with convective boundary conditions in manufacturing, utilizing an adaptive loss normalization scheme and physics-informed feature engineering to achieve accurate predictions [23]. Bo Zhang, et al. (2024) presented an evolutionary computation method for automatically discovering superior PINN models by concurrently searching for optimal network structures and novel parametric activation functions, which demonstrates superior approximation accuracy, faster convergence, and commendable generalization performance in solving various PDEs compared to traditional search methods [20].

Hansaem Oh, et al. (2025) presented a PINN-based method to solve the heat transfer equation under imperfect contact conditions by employing an axis-augmentation technique that extends discontinuous solutions to a continuous function in an augmented variable space [44]. Bischof, et al. (2025) presented a novel self-adaptive loss balancing scheme for PINNs named ReLoBRaLo (Relative Loss Balancing with Random Lookback), designed to effectively weight multiple competitive loss functions and mitigate gradient pathologies arising from imbalanced loss terms, thereby improving PINN training effectiveness, accuracy, and computational efficiency for solving both forward and inverse problems across various PDEs [46]. Kalyan, et al. (2025) introduced a novel PINN framework for the numerical simulation of transient heat transfer involving a moving heat source by proposing a new training method that employs continuous time-stepping through transfer learning, enabling the computation of large temporal intervals efficiently without increasing network complexity [19]. Di He, et al. (2025) presented a systematic summary of the limitations of PINNs, identifying three root causes for their failure in solving PDEs, and outlining future directions and prospects for the intelligent computing of PDEs [21]. Most recently, Bueno, et al. (2025) presented a convergence analysis of PINN, investigating the influence of discretization

and network architecture on approximation and estimation errors, and compares their performance with FDM for the 2D heat equation, demonstrating PINN’s advantage in approximating solutions for all time steps simultaneously without error propagation [47].

To aid in clarity and highlight contributions, Table 2.1 presents a comprehensive summary.

Table 2.1: Summary of key contributions of related works

Year	Collective Research Highlights
2010–2019	Foundational work on neural network approaches for solving PDEs. Raissi et al. proposed the original PINN framework, which integrates physics constraints into neural network loss functions [3]. This paved the advancement for novel approaches to enhance PINNs for complex problems and set the stage for further developments.
2020	Initial build-up focused on PINN’s stability and efficiency. Lu Lu et al. introduced DeepXDE as a standalone library [6].
2021	Concerns to instability and convergence of PINNs. Wang, et al elaborated the failure of PINN via numerical stiffness and proposed novel approaches to mitigate pathologies [37]. Krishapriyan, et al. characterized failure modes in PINNs [32].
2022	The focus to robustness and applicability. Aygun et al. attempted thermal convection problems [43]. Xing, et al. introduced a Hybrid approach that combines FDM and PINN. These developments improved the reliability of PINNs for more complex systems [27].
2023	The potential of practical significance of PINNs were studied. Xing, et al. designed PINN for 3D anisotropic heat conduction problem [27]. Savovic, et al. compared explicit scheme to PINN for Burger’s Equation [41].
2024	The priority to make PINN a better method of solving PDE emerged. Bo Zhang, et al. presented evolutionary computation method to search for superior PINNs [20]. Zobiery, et al. solved conductive heat transfer with convective boundary conditions [23]. Bhatnagar, et al. demonstrated capability of PINN to solve Navier-Stokes equation at high Reynolds number [26].
2025	Recent contributions: Kalyan et al. have examined PINN for moving heat problem [19]. Bischof, et al. presented a novel self-adaptive loss balancing scheme. Hansaem Oh, et al. presented a PINN-based method to solve the heat transfer equation under imperfect conditions. Bueno, et al presented comparison of FDM for 2D heat equation demonstrating PINN’s advantage [47].

2.4 Research Gaps

To the best of our knowledge, no existing study has comprehensively benchmarked PINNs against Explicit, Implicit, and Crank-Nicolson FDM schemes for solving 1D, 2D, and 3D heat equations under coarse discretization. Prior comparisons often exclude 3D cases and rarely account for GPU/CPU performance parity. Furthermore, detailed evaluations of accuracy, runtime, and hyperparameter sensitivity across different spatial dimensions remain largely unexplored. While FDMs are well-regarded for their accuracy and computational efficiency, and PINNs offer a flexible, mesh-free alternative for solving PDEs, a rigorous, side-by-side assessment of these methods for this case is lacking. This gap motivates the present work, which aims to establish a systematic benchmarking framework.

CHAPTER 3

Mathematical Framework

In this chapter, we detail the mathematical concept implemented to solve and simulate in one, two, and three dimensions using both FDM schemes and PINN. The mathematical concepts presented, aims to act as an establishment for this project, and directs the flow of methods implemented to shape it towards the objective.

3.1 Analytical Solutions

The following descriptions of analytical solutions for each dimensional heat equations are stated referring to the works of Lawrence Craig Evans [1], Horatio Scott Carslaw, et al [5], and Walter Alexander Strauss [8].

3.1.1 Analytical Solution of 1D Heat Equation

Analytical solutions to the 1D heat equation are derived using separation of variables, decomposing the solution into spatial and temporal components, and Fourier series, which represent the solution as a sum of sine functions. For Dirichlet boundary conditions $u(0, t) = u(L, t) = 0$ and initial condition $u(x, 0) = u_0(x)$, the solution is:

$$u(x, t) = \sum_{n=1}^{\infty} b_n \sin\left(\frac{n\pi x}{L}\right) \exp\left[-\alpha \left(\frac{n\pi}{L}\right)^2 t\right], \quad (3.1)$$

where $b_n = \frac{2}{L} \int_0^L u_0(x) \sin\left(\frac{n\pi x}{L}\right) dx$ are Fourier coefficients determined by the initial temperature profile.

3.1.2 Analytical Solution of 2D Heat Equation

The 2D heat equation on $[0, L]^2$ with Dirichlet conditions $u = 0$ on boundaries and initial condition $u(x, y, 0) = u_0(x, y)$ is solved by separation of variables, assuming $u(x, y, t) = X(x)Y(y)T(t)$, yielding three ordinary differential equations. The solution

is:

$$u(x, y, t) = \sum_{m=1}^{\infty} \sum_{n=1}^{\infty} A_{mn} \sin\left(\frac{m\pi x}{L}\right) \sin\left(\frac{n\pi y}{L}\right) \exp(-\alpha \lambda_{mn}^2 t), \quad (3.2)$$

where

$$\lambda_{mn}^2 = \pi^2 \left(\frac{m^2}{L^2} + \frac{n^2}{L^2} \right), \quad (3.3)$$

and $A_{mn} = \frac{4}{L^2} \int_0^L \int_0^L u_0(x, y) \sin\left(\frac{m\pi x}{L}\right) \sin\left(\frac{n\pi y}{L}\right) dx dy$ are double Fourier coefficients.

3.1.3 Analytical Solution of 3D Heat Equation

For the 3D heat equation on $[0, L]^3$ with Dirichlet conditions $u = 0$ on boundaries and initial condition $u(x, y, z, 0) = u_0(x, y, z)$, separation of variables yields:

$$u(x, y, z, t) = \sum_{m=1}^{\infty} \sum_{n=1}^{\infty} \sum_{p=1}^{\infty} A_{mnp} \sin\left(\frac{m\pi x}{L}\right) \sin\left(\frac{n\pi y}{L}\right) \sin\left(\frac{p\pi z}{L}\right) \exp(-\alpha \lambda_{mnp}^2 t), \quad (3.4)$$

where

$$\lambda_{mnp}^2 = \pi^2 \left(\frac{m^2}{L^2} + \frac{n^2}{L^2} + \frac{p^2}{L^2} \right), \quad (3.5)$$

and $A_{mnp} = \frac{8}{L^3} \int_0^L \int_0^L \int_0^L u_0(x, y, z) \sin\left(\frac{m\pi x}{L}\right) \sin\left(\frac{n\pi y}{L}\right) \sin\left(\frac{p\pi z}{L}\right) dx dy dz$ are Fourier coefficients.

3.2 Finite Difference Method

FDM approximates partial differential equations by discretizing derivatives using differences between grid points. For the heat equation on their respective dimensions, the spatial and temporal domains are divided into grids with steps Δx , Δy , Δz , and Δt . The following mathematical descriptions are stated referring to the works of Randall J. LeVeque [2], G.D. Smith [13], Gerald Recktenwald [16], John C. Strikwerda [48], Richard H. Pletcher, et al. [49], and Keith William Morton, et al. [33].

3.2.1 Explicit Scheme

The explicit scheme uses forward differencing in time and central differencing in space. For the 1D heat equation:

$$u_i^{n+1} = u_i^n + r(u_{i+1}^n - 2u_i^n + u_{i-1}^n), \quad (3.6)$$

where $r = \alpha \Delta t / \Delta x^2$, $u_i^n = u(x_i, t_n)$, $x_i = i\Delta x$, and $t_n = n\Delta t$.

For 2D heat equation:

$$u_{i,j}^{n+1} = u_{i,j}^n + r_x(u_{i+1,j}^n - 2u_{i,j}^n + u_{i-1,j}^n) + r_y(u_{i,j+1}^n - 2u_{i,j}^n + u_{i,j-1}^n), \quad (3.7)$$

where $r_x = \alpha\Delta t/\Delta x^2$, $r_y = \alpha\Delta t/\Delta y^2$.

For 3D heat equation:

$$\begin{aligned} u_{i,j,k}^{n+1} = & u_{i,j,k}^n + r_x(u_{i+1,j,k}^n - 2u_{i,j,k}^n + u_{i-1,j,k}^n) \\ & + r_y(u_{i,j+1,k}^n - 2u_{i,j,k}^n + u_{i,j-1,k}^n) \\ & + r_z(u_{i,j,k+1}^n - 2u_{i,j,k}^n + u_{i,j,k-1}^n), \end{aligned} \quad (3.8)$$

where $r_z = \alpha\Delta t/\Delta z^2$.

Stability requires the Courant-Friedrichs-Lewy (CFL) condition [34]:

$$r \leq \frac{1}{2}(1D), \quad r_x + r_y \leq \frac{1}{2}(2D), \quad r_x + r_y + r_z \leq \frac{1}{2}(3D). \quad (3.9)$$

3.2.2 Implicit Scheme

The implicit scheme uses backward differencing in time.

For 1D heat equation:

$$u_i^{n+1} - r(u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}) = u_i^n, \quad (3.10)$$

forming a tridiagonal system $A\mathbf{u}^{n+1} = \mathbf{u}^n$, where A has diagonal $1 + 2r$, off-diagonals $-r$.

For 2D heat equation:

$$\begin{aligned} u_{i,j}^{n+1} - r_x(u_{i+1,j}^{n+1} - 2u_{i,j}^{n+1} + u_{i-1,j}^{n+1}) \\ - r_y(u_{i,j+1}^{n+1} - 2u_{i,j}^{n+1} + u_{i,j-1}^{n+1}) = u_{i,j}^n, \end{aligned} \quad (3.11)$$

with sparse matrix (for a vectorized solution ' $\mathbf{u}^{n+1}$ '):

$$A_{\text{implicit}} = I - \alpha\Delta t \left(\frac{D_{2x}}{\Delta x^2} \otimes I_{N_y} + I_{N_x} \otimes \frac{D_{2y}}{\Delta y^2} \right), \quad (3.12)$$

where D_{2x} , D_{2y} are second-difference matrices, and I_{N_x} , I_{N_y} are identity matrices of size $N_x \times N_x$ and $N_y \times N_y$ respectively (where N_x and N_y are the number of grid points in the x and y directions).

For 3D heat equation:

$$\begin{aligned} u_{i,j,k}^{n+1} - r_x(u_{i+1,j,k}^{n+1} - 2u_{i,j,k}^{n+1} + u_{i-1,j,k}^{n+1}) \\ - r_y(u_{i,j+1,k}^{n+1} - 2u_{i,j,k}^{n+1} + u_{i,j-1,k}^{n+1}) \\ - r_z(u_{i,j,k+1}^{n+1} - 2u_{i,j,k}^{n+1} + u_{i,j,k-1}^{n+1}) = u_{i,j,k}^n, \end{aligned} \quad (3.13)$$

with matrix (for a vectorized solution ‘ $\mathbf{u}^{n+1}$ ’):

$$A_{\text{implicit}} = I - \alpha \Delta t \left(\frac{D_{2x}}{\Delta x^2} \otimes I_{N_y} \otimes I_{N_z} + I_{N_x} \otimes \frac{D_{2y}}{\Delta y^2} \otimes I_{N_z} + I_{N_x} \otimes I_{N_y} \otimes \frac{D_{2z}}{\Delta z^2} \right), \quad (3.14)$$

solved using sparse linear solvers. Here, $I_{N_x}, I_{N_y}, I_{N_z}$ are identity matrices corresponding to the number of grid points in each direction. It is unconditionally stable.

3.2.3 Crank-Nicolson Scheme

The Crank-Nicolson scheme averages explicit and implicit methods.

For 1D heat equation:

$$\begin{aligned} u_i^{n+1} - \frac{r}{2}(u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}) \\ = u_i^n + \frac{r}{2}(u_{i+1}^n - 2u_i^n + u_{i-1}^n), \end{aligned} \quad (3.15)$$

forming a tridiagonal system $A\mathbf{u}^{n+1} = B\mathbf{u}^n + \mathbf{b}$.

For 2D heat equation:

$$\begin{aligned} u_{i,j}^{n+1} - \frac{r_x}{2}(u_{i+1,j}^{n+1} - 2u_{i,j}^{n+1} + u_{i-1,j}^{n+1}) \\ - \frac{r_y}{2}(u_{i,j+1}^{n+1} - 2u_{i,j}^{n+1} + u_{i,j-1}^{n+1}) \\ = u_{i,j}^n + \frac{r_x}{2}(u_{i+1,j}^n - 2u_{i,j}^n + u_{i-1,j}^n) \\ + \frac{r_y}{2}(u_{i,j+1}^n - 2u_{i,j}^n + u_{i,j-1}^n), \end{aligned} \quad (3.16)$$

with matrices (for a vectorized solution):

$$\begin{aligned} A_{\text{cn}} &= I - 0.5\alpha\Delta t \left(\frac{D_{2x}}{\Delta x^2} \otimes I_{N_y} + I_{N_x} \otimes \frac{D_{2y}}{\Delta y^2} \right), \\ B_{\text{cn}} &= I + 0.5\alpha\Delta t \left(\frac{D_{2x}}{\Delta x^2} \otimes I_{N_y} + I_{N_x} \otimes \frac{D_{2y}}{\Delta y^2} \right), \end{aligned} \quad (3.17)$$

For 3D heat equation:

$$\begin{aligned}
& u_{i,j,k}^{n+1} - \frac{r_x}{2}(u_{i+1,j,k}^{n+1} - 2u_{i,j,k}^{n+1} + u_{i-1,j,k}^{n+1}) \\
& - \frac{r_y}{2}(u_{i,j+1,k}^{n+1} - 2u_{i,j,k}^{n+1} + u_{i,j-1,k}^{n+1}) \\
& - \frac{r_z}{2}(u_{i,j,k+1}^{n+1} - 2u_{i,j,k}^{n+1} + u_{i,j,k-1}^{n+1}) \\
& = u_{i,j,k}^n + \frac{r_x}{2}(u_{i+1,j,k}^n - 2u_{i,j,k}^n + u_{i-1,j,k}^n) \\
& + \frac{r_y}{2}(u_{i,j+1,k}^n - 2u_{i,j,k}^n + u_{i,j-1,k}^n) \\
& + \frac{r_z}{2}(u_{i,j,k+1}^n - 2u_{i,j,k}^n + u_{i,j,k-1}^n),
\end{aligned} \tag{3.18}$$

with matrices (for a vectorized solution):

$$\begin{aligned}
A_{\text{cn}} &= I - 0.5\alpha\Delta t \left(\frac{D_{2x}}{\Delta x^2} \otimes I_{N_y} \otimes I_{N_z} \right. \\
& \quad \left. + I_{N_x} \otimes \frac{D_{2y}}{\Delta y^2} \otimes I_{N_z} + I_{N_x} \otimes I_{N_y} \otimes \frac{D_{2z}}{\Delta z^2} \right), \\
B_{\text{cn}} &= I + 0.5\alpha\Delta t \left(\frac{D_{2x}}{\Delta x^2} \otimes I_{N_y} \otimes I_{N_z} \right. \\
& \quad \left. + I_{N_x} \otimes \frac{D_{2y}}{\Delta y^2} \otimes I_{N_z} + I_{N_x} \otimes I_{N_y} \otimes \frac{D_{2z}}{\Delta z^2} \right),
\end{aligned} \tag{3.19}$$

solved using sparse linear solvers (where $\otimes$ represents the Kronecker product). It is second-order accurate and unconditionally stable.

3.3 PINN Working Mechanism

As prior mention, PINNs approximate PDE solutions by training a neural network to satisfy the PDE, boundary, and initial conditions [3]. The loss function is [3, 4, 19]:

$$\mathcal{L} = \lambda_{\text{PDE}}\mathcal{L}_{\text{PDE}} + \lambda_{\text{BC}}\mathcal{L}_{\text{BC}} + \lambda_{\text{IC}}\mathcal{L}_{\text{IC}}, \tag{3.20}$$

where:

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_p} \sum_{i=1}^{N_p} \left| \frac{\partial u}{\partial t}(x_i, t_i) - \alpha \Delta u(x_i, t_i) \right|^2, \tag{3.21}$$

$$\mathcal{L}_{\text{BC}} = \frac{1}{N_b} \sum_{i=1}^{N_b} |u(x_i, t_i) - 0|^2, \tag{3.22}$$

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_i} \sum_{i=1}^{N_i} |u(x_i, 0) - u_0(x_i)|^2, \tag{3.23}$$

enforce the heat equation, Dirichlet boundary conditions, and initial condition, respectively. Derivatives are computed via automatic differentiation, a technique that

calculates exact derivatives of the neural network’s output using chain rules, avoiding numerical discretization [3, 15].

PINNs solve the heat equation by integrating PDE constraints into a neural network’s loss function, as depicted in the Figure 3.1. The process begins with defining initial conditions $u_0(x)$, boundary conditions, and thermal diffusivity (α). Spatial (x_i) and temporal (t_j) collocation points are sampled, often using LHS [14]. A feedforward neural network, parameterized by θ (weights and biases), approximates the solution $u_\theta(x, t)$ [3]. Automatic differentiation computes derivatives at collocation points [15]. Three loss terms are evaluated: the PDE loss ($\mathcal{L}_{\text{PDE}}$), enforcing the heat equation; the boundary loss ($\mathcal{L}_{\text{BC}}$), ensuring boundary conditions; and the initial loss ($\mathcal{L}_{\text{IC}}$), matching initial conditions. These are summed to form the total loss $\mathcal{L}(\theta) = \lambda_{\text{PDE}}\mathcal{L}_{\text{PDE}} + \lambda_{\text{BC}}\mathcal{L}_{\text{BC}} + \lambda_{\text{IC}}\mathcal{L}_{\text{IC}}$ (where λ_{PDE} , λ_{BC} , & λ_{IC} are the respective weights) [3, 4]. Backpropagation calculates gradients, and an optimizer (Adam or Adam+L-BFGS) updates θ [22, 37]. If the loss is above a threshold (ϵ) or maximum epochs are not reached, training continues via a loop; otherwise, the trained solution $u_\theta(x, t)$ is output [3].

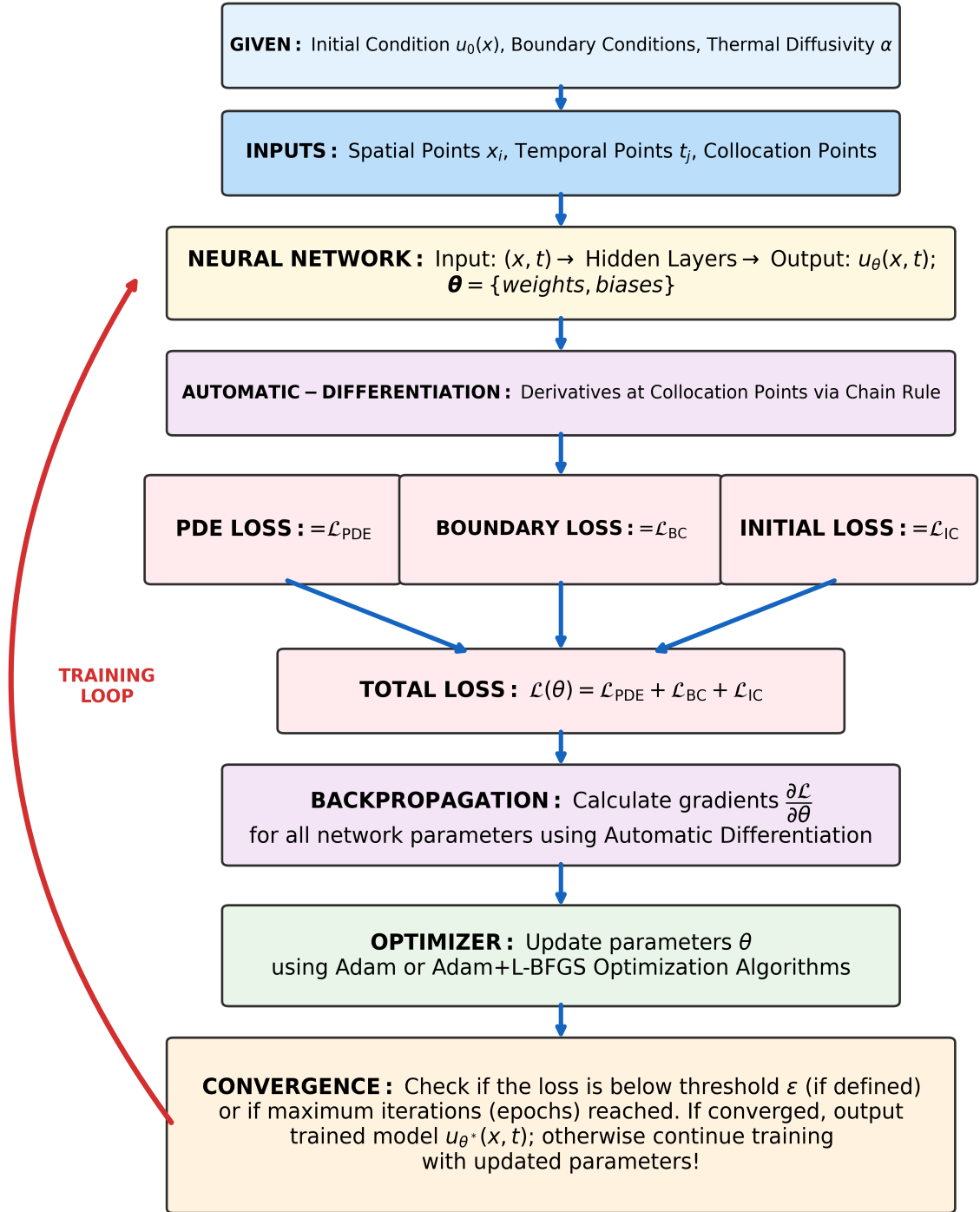


Figure 3.1: Flowchart representing PINN's working mechanism

CHAPTER 4

Methodology

In this chapter, we detail the methodology adopted for simulating and benchmarking the heat equation in one, two, and three dimensions using both FDM schemes and PINN. The computational setup, simulation parameters, and analytical formulations are systematically presented to ensure readability and clarity.

4.1 Computational Environment

The project was implemented using Python programming language. The interpreter used was Kaggle Notebook, an online platform, that offered GPU usage of 12 hours on a continuous run and a total of 30 hours per week before resetting. The overall description of the capacity and limit of the interpreter is described as in Table 4.1:

Table 4.1: Hardware specifications for kaggle notebook environment

Component	Core Count	Memory	Hours/Week
Intel Xeon CPU 2.00 GHz	4 vCPU cores	32 GB RAM	Unlimited
NVIDIA Tesla P100 GPU 6	3584 CUDA cores	16 GB VRAM	30 hours per week
Storage	—	20 GB	Unlimited

4.2 Simulation Setup

The heat equation was solved in 1D, 2D, and 3D domains to model heat diffusion in a copper material [1, 5]. The thermal diffusivity was set to reflect copper at 25°C:

$$\alpha = 0.00011644 \text{ m}^2 \text{ s}^{-1}, \quad (4.1)$$

consistent with literature values for copper thin films and bulk copper at room temperature [9, 11], and, the domain length for each dimension was:

$$L = 1.0 \text{ m.} \quad (4.2)$$

Spatial grids were defined with $Nx = Ny = Nz = 15$ points, resulting in spatial steps:

$$\Delta x = \Delta y = \Delta z = \frac{L}{Nx - 1} \approx 0.0715 \text{ m,} \quad (4.3)$$

for 1D, 2D, and 3D, respectively. The time step was:

$$\Delta t = 0.6 \text{ s,} \quad (4.4)$$

with a final simulation time:

$$t_{\text{final}} = 60 \text{ s.} \quad (4.5)$$

The total number of time steps was computed as:

$$n_{\text{steps}} = \left\lfloor \frac{t_{\text{final}}}{\Delta t} \right\rfloor = 100. \quad (4.6)$$

Solutions were evaluated at 60 time points:

$$t = [1.0, 2.0, \dots, 60.0] \text{ s,} \quad (4.7)$$

generated using a linear spacing function. Spatial grids were defined as:

$$x = [0, \Delta x, \dots, L], \quad y = [0, \Delta y, \dots, L], \quad z = [0, \Delta z, \dots, L], \quad (4.8)$$

with 2D and 3D meshes created using:

$$(X, Y) = \text{meshgrid}(x, y), \quad (X_{3D}, Y_{3D}, Z_{3D}) = \text{meshgrid}(x, y, z). \quad (4.9)$$

The stability parameter for FDM was computed for each dimension:

$$r = \frac{\alpha \Delta t}{(\Delta x)^2}, \quad r_x + r_y = \frac{\alpha \Delta t}{(\Delta x)^2} + \frac{\alpha \Delta t}{(\Delta y)^2}, \quad r_x + r_y + r_z = \frac{\alpha \Delta t}{(\Delta x)^2} + \frac{\alpha \Delta t}{(\Delta y)^2} + \frac{\alpha \Delta t}{(\Delta z)^2}, \quad (4.10)$$

to ensure stability for explicit schemes in accordance with the Courant-Friedrichs-Lewy (CFL) condition for explicit finite difference schemes [8, 34].

The triangular initial conditions were defined as [2, 5]:

For 1D:

$$u(x, 0) = \begin{cases} \frac{2x}{L}, & 0 \leq x \leq \frac{L}{2}, \\ \frac{2(L-x)}{L}, & \frac{L}{2} < x \leq L, \end{cases} \quad (4.11)$$

For 2D:

$$u(x, y, 0) = \begin{cases} \frac{4xy}{L^2}, & 0 \leq x \leq \frac{L}{2}, 0 \leq y \leq \frac{L}{2}, \\ \frac{4x(L-y)}{L^2}, & 0 \leq x \leq \frac{L}{2}, \frac{L}{2} < y \leq L, \\ \frac{4(L-x)y}{L^2}, & \frac{L}{2} < x \leq L, 0 \leq y \leq \frac{L}{2}, \\ \frac{4(L-x)(L-y)}{L^2}, & \frac{L}{2} < x \leq L, \frac{L}{2} < y \leq L, \end{cases} \quad (4.12)$$

For 3D:

$$u(x, y, z, 0) = \begin{cases} \frac{8xyz}{L^3}, & 0 \leq x, y, z \leq \frac{L}{2}, \\ \frac{8xy(L-z)}{L^3}, & 0 \leq x, y \leq \frac{L}{2}, \frac{L}{2} < z \leq L, \\ \frac{8x(L-y)z}{L^3}, & 0 \leq x, z \leq \frac{L}{2}, \frac{L}{2} < y \leq L, \\ \frac{8x(L-y)(L-z)}{L^3}, & 0 \leq x \leq \frac{L}{2}, \frac{L}{2} < y, z \leq L, \\ \frac{8(L-x)yz}{L^3}, & \frac{L}{2} < x \leq L, 0 \leq y, z \leq \frac{L}{2}, \\ \frac{8(L-x)y(L-z)}{L^3}, & \frac{L}{2} < x, z \leq L, 0 \leq y \leq \frac{L}{2}, \\ \frac{8(L-x)(L-y)z}{L^3}, & \frac{L}{2} < x, y \leq L, 0 \leq z \leq \frac{L}{2}, \\ \frac{8(L-x)(L-y)(L-z)}{L^3}, & \frac{L}{2} < x, y, z \leq L, \end{cases} \quad (4.13)$$

with Dirichlet boundary conditions [1, 5]:

$$u = 0 \text{ on all domain boundaries}, \quad (4.14)$$

applied by setting the temperature to zero at domain edges (e.g., $u(0, t) = u(L, t) = 0$ for 1D, and similarly for 2D and 3D boundaries). Initial conditions were evaluated on the grids and boundary conditions applied to produce initial fields $u_{i,1D}$, $u_{i,2D}$, and $u_{i,3D}$.

4.3 Analytical Solutions

Analytical solutions for the 1D, 2D, and 3D heat equations were computed using separation of variables and Fourier series, as detailed in equations (3.1), (3.2), (3.4). The solutions below were directly implemented to the code [1, 5]:

$$u(x, t) = \sum_{n=1}^{50} b_n e^{-\alpha \left(\frac{n\pi}{L}\right)^2 t} \sin\left(\frac{n\pi x}{L}\right), \quad (\text{for 1D}), \quad (4.15)$$

$$u(x, y, t) = \sum_{m=1}^{50} \sum_{n=1}^{50} A_{mn} e^{-\alpha \lambda_{mn}^2 t} \sin\left(\frac{m\pi x}{L}\right) \sin\left(\frac{n\pi y}{L}\right), \quad (\text{for 2D}), \quad (4.16)$$

$$u(x, y, z, t) = \sum_{m=1}^{50} \sum_{n=1}^{50} \sum_{p=1}^{50} A_{mnp} e^{-\alpha \lambda_{mnp}^2 t} \sin\left(\frac{m\pi x}{L}\right) \sin\left(\frac{n\pi y}{L}\right) \sin\left(\frac{p\pi z}{L}\right), \quad (\text{for 3D}), \quad (4.17)$$

where the Fourier coefficients b_n , A_{mn} , and A_{mnp} are determined from the respective initial conditions (4.11), (4.12), (4.13). The series were truncated at $N = 50$ terms, ensuring a balance between accuracy and computational cost. These solutions were then evaluated at time points in (4.7) and at spatial grid points defined in (4.8), and stored in arrays for comparison with the numerical schemes.

The process involved:

1. Computing the solution for each evaluation time t .
2. Iterating over grid points (x for 1D, (x, y) for 2D, (x, y, z) for 3D).
3. Saving results to CSV files with columns $[t, x, \text{temperature}]$ for 1D, $[t, x, y, \text{temperature}]$ for 2D, and $[t, x, y, z, \text{temperature}]$ for 3D, using a data frame structure.

4.4 Implementation of FDM Schemes

FDMs were implemented with explicit, implicit, and Crank-Nicolson schemes for 1D, 2D, and 3D heat equations.

4.4.1 Explicit Scheme

The explicit scheme (3.6), (3.7), (3.8) was applied by:

1. Initializing fields with conditions (4.11), (4.12), (4.13).

2. Iterating over n_{steps} (4.6), updating interior points using the respective scheme for each dimension.
3. Applying boundary conditions (4.14) at each time step.
4. Saving solutions at evaluation times (4.7) to CSV files, structured as in Section 3.2.

4.4.2 Implicit Scheme

The implicit scheme (3.10), (3.11), (3.13) was implemented by:

1. Initializing fields with conditions (4.11), (4.12), (4.13).
2. Constructing sparse matrices (3.12), (3.14) for 2D and 3D, or tridiagonal matrix for 1D.
3. Iterating over n_{steps} , solving the system $A_{\text{implicit}} \mathbf{u}^{n+1} = \mathbf{u}^n$ using the Thomas algorithm (1D) or sparse linear solvers (2D, 3D).
4. Applying boundary conditions (4.14) after each solve.
5. Saving solutions at evaluation times to CSV files.

4.4.3 Crank-Nicolson Scheme

The Crank-Nicolson scheme (3.15), (3.16), (3.18) was implemented by:

1. Initializing fields with conditions (4.11), (4.12), (4.13).
2. Constructing sparse matrices (3.17), (3.19) for 2D and 3D, or tridiagonal matrices for 1D.
3. Iterating over n_{steps} , solving $A_{\text{cn}} \mathbf{u}^{n+1} = B_{\text{cn}} \mathbf{u}^n$ using the Thomas algorithm (1D) or sparse linear solvers (2D, 3D).
4. Applying boundary conditions (4.14) after each solve.
5. Saving solutions at evaluation times to CSV files.

4.5 Implementation of PINN

PINNs were implemented using a feedforward neural network [3]. The PINN architecture was defined as:

Table 4.2: Network architecture and hyperparameters

Component	Specification
Input layer	d neurons
Hidden layers	h layers, w neurons each
Output layer	1 neuron
Input dimension (d)	$\begin{cases} 2 & (1\text{D: } x, t) \\ 3 & (2\text{D: } x, y, t) \\ 4 & (3\text{D: } x, y, z, t) \end{cases}$
Hyperparameter ranges	1D: $h \in \{2\}$, $w \in \{4, 6, 8\}$ 2D: $h \in \{4, 6\}$, $w \in \{32, 64\}$ 3D: $h \in \{6, 8\}$, $w \in \{128\}$ Learning rate $\in \{5 \times 10^{-3}, 5 \times 10^{-4}\}$
Activation functions	tanh, GELU, SiLU, & Mish
Optimizers	Adam & Adam+LBFGS
Weight initialization	Xavier uniform & Kaiming normal

Weights were initialized using Xavier uniform initialization and Kaiming normal initialization [35]. Inputs were normalized to $[-1, 1]$ using domain bounds $[0, L]$ for spatial coordinates and $[0, t_{\text{final}}]$ for time [3]:

$$x_{\text{norm}} = 2 \frac{x - x_{\min}}{x_{\max} - x_{\min}} - 1 \quad (4.18)$$

The loss function applied was [3, 4]:

$$\mathcal{L} = \lambda_{\text{PDE}} \mathcal{L}_{\text{PDE}} + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}} + \lambda_{\text{IC}} \mathcal{L}_{\text{IC}}, \quad (4.19)$$

with components:

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_p} \sum_{i=1}^{N_p} \left| \frac{\partial u}{\partial t}(x_i, t_i) - \alpha \Delta u(x_i, t_i) \right|^2, \quad (4.20)$$

$$\mathcal{L}_{\text{BC}} = \frac{1}{N_b} \sum_{i=1}^{N_b} |u(x_i, t_i) - 0|^2, \quad (4.21)$$

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_i} \sum_{i=1}^{N_i} |u(x_i, 0) - u_0(x_i)|^2, \quad (4.22)$$

where $\Delta u = u_{xx}$ (1D), $u_{xx} + u_{yy}$ (2D), or $u_{xx} + u_{yy} + u_{zz}$ (3D), and u_0 matches equations (4.11), (4.12), (4.13).

Collocation points were generated using LHS [14] covering domains $[0, L] \times [0, t_{\text{final}}]$ (1D), $[0, L]^2 \times [0, t_{\text{final}}]$ (2D), and $[0, L]^3 \times [0, t_{\text{final}}]$ (3D).

Table 4.3: Collocation points distribution by dimension

Dimension	PDE Points (N_p)	Boundary Points (N_b)	Initial Points (N_i)
1D	3,000	1,000	2,000
2D	30,000	10,000	20,000
3D	60,000	20,000	40,000

Boundary points were sampled on domain edges (e.g., $x = 0, L$ for 1D), and initial points at $t = 0$. Derivatives were computed using automatic differentiation via PyTorch’s autograd, applying the chain rule to calculate exact gradients without numerical discretization [3, 15].

4.5.1 Latin Hypercube Sampling

The collocation points for training the PINNs were generated using Latin Hypercube Sampling (LHS), a space-filling experimental design technique [14]. LHS divides each input dimension into equally probable intervals and samples exactly once from each interval, ensuring better coverage than random sampling. This sampling strategy proved particularly effective for capturing the solution behavior across the spatiotemporal domains while maintaining computational efficiency [14]. The distribution of collocation points using LHS for one-dimensional, two-dimensional, and three-dimensional space is depicted in Figures 4.1, 4.2, and 4.3 respectively.

4.5.2 Training

Training used three loss weighting strategies, detailed in Table 4.4.

Table 4.4: Loss weighting strategies for PINN training

Strategy	Features	Description
Equal Weighting	Uniform weights: $\lambda_{\text{PDE}} = \lambda_{\text{BC}} = \lambda_{\text{IC}} = 1$	Applies equal importance to PDE, boundary, and initial condition losses [3].
Adaptive GradNorm	Dynamic weights: $\lambda_i = \frac{\text{Total Norm}}{3 \cdot \text{GradNorm}_i}$ Moving Average = 0.9	Balances loss terms by scaling weights inversely to gradient norms, ensuring stable training [37]. Weights are updated using exponential moving average of gradient norms.
Adaptive LR Annealing	Dynamic weights: $\lambda_{\text{PDE}} = \mathcal{L}_{\text{PDE}} / \mathcal{L}_{\text{total}},$ $\lambda_{\text{BC}} = \mathcal{L}_{\text{BC}} / \mathcal{L}_{\text{total}},$ $\lambda_{\text{IC}} = \mathcal{L}_{\text{IC}} / \mathcal{L}_{\text{total}}$	Adjusts weights based on relative loss magnitudes, emphasizing larger components to improve convergence [7].

Optimizers included Adam with learning rate 5×10^{-3} or 5×10^{-4} , and Adam+LBFGS (switching to LBFGS at 90% of total epochs). Training ran for 6,000 epochs with gradient clipping applied to stabilize optimization. Solutions were evaluated at grid points (4.8) and times (4.7), and saved to CSV files.

A `PINNTuner` class managed the hyperparameter search, systematically testing combinations across depths, widths, activations, learning rates, optimizers, weight initializations, loss weighting strategies, and input normalization.

The tuner supports:

- Dimension-specific configurations (1D, 2D, 3D)
- Adaptive sampling based on dimension
- Progress tracking and checkpointing
- Automatic result saving and resumption
- Reproducibility via fixed random seeds

4.6 Performance Evaluation

Solutions were evaluated using error metrics [3, 6]:

$$\text{L2 error} = \sqrt{\sum_{i=1}^N (u_{\text{pred},i} - u_{\text{exact},i})^2}, \quad (4.23)$$

$$\text{L-infinity error} = \max_i |u_{\text{pred},i} - u_{\text{exact},i}|, \quad (4.24)$$

$$\text{Relative L2 error} = \frac{\sqrt{\sum_{i=1}^N (u_{\text{pred},i} - u_{\text{exact},i})^2}}{\sqrt{\sum_{i=1}^N u_{\text{exact},i}^2}}, \quad (4.25)$$

where u_{pred} represents predicted values from PINN, and u_{exact} denotes analytical solutions used as benchmarks. Errors (4.23), (4.24), and (4.25) were computed by comparing numerical solutions to analytical solutions at evaluation times. Execution time was measured using `time.perf_counter()`, and GPU memory usage was tracked via `torch.cuda.max_memory_allocated()`, with results stored in CSV files. Loss histories (PDE, BC, IC, and total losses) were visualized on logarithmic scale to monitor training convergence across all epochs.

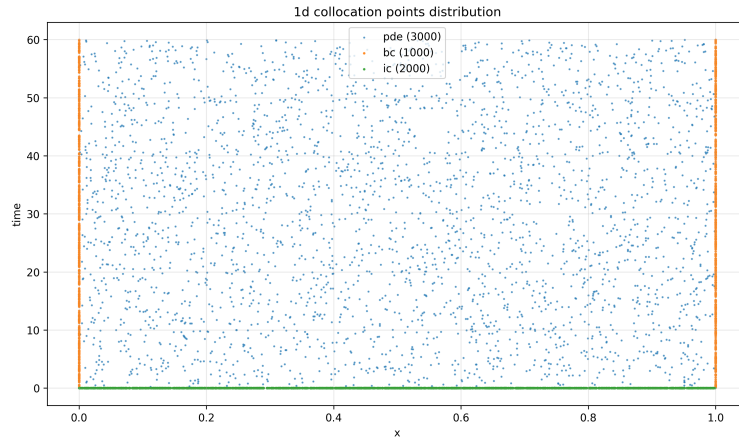


Figure 4.1: Distribution of collocation points for 1D heat equation in a rod.

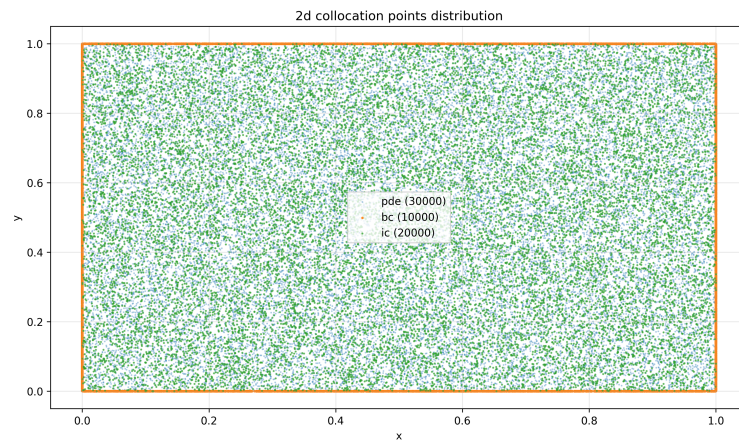


Figure 4.2: Distribution of collocation points for 2D heat equation in a sheet.

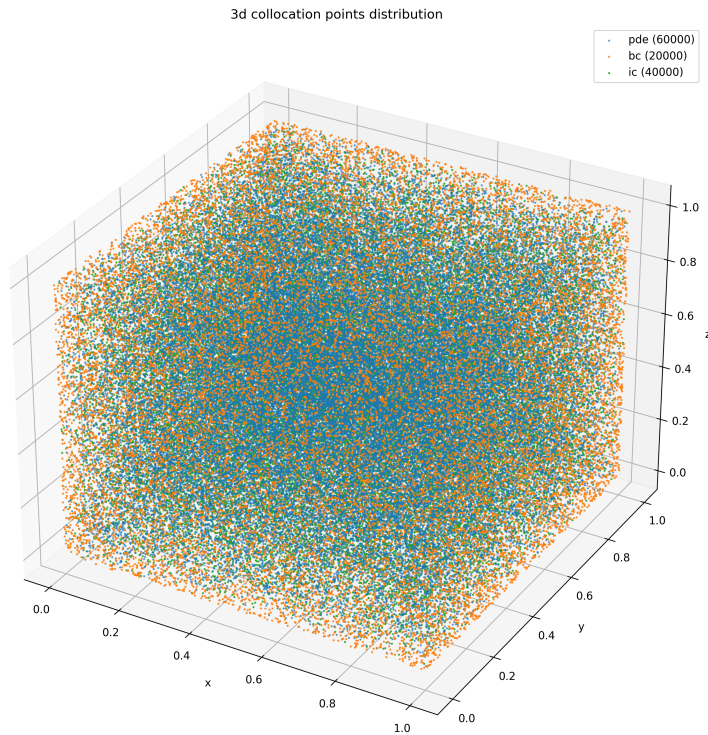


Figure 4.3: Distribution of collocation points for 3D heat equation in a cube.

CHAPTER 5

Results & Discussion

This chapter presents the empirical findings of the study, organized by dimension and evaluation criteria. The results focus on identifying optimal PINN architectures, comparing performance with traditional FDM schemes, and analyzing scalability across 1D, 2D, and 3D heat equations. Emphasis is placed on accuracy, computational efficiency, and hardware utilization to assess the practical viability of each method.

5.1 Optimal Configurations

After evaluating all the experimental configurations across all dimensions, the best performing setups for each dimension were identified:

Table 5.1: Optimal PINN configurations by dimension

Hyperparameter	1D	2D	3D
Layers	2	6	6
Neurons per Layer	8	64	128
Activation	Tanh	Tanh	Tanh
Learning Rate	5×10^{-3}	5×10^{-4}	5×10^{-4}
Optimizer	Adam	Adam	Adam
Initialization	Kaiming Normal	Xavier Uniform	Xavier Uniform
Weighting	Adaptive GradNorm	Adaptive GradNorm	Adaptive GradNorm

These configurations consistently employed the Tanh activation function and Adaptive GradNorm weighting strategy across all dimensions, demonstrating their effectiveness for physics-informed learning. The architectural complexity scaled with dimension: 1D required minimal depth (2 layers, 8 neurons), while 3D demanded significantly larger networks (6 layers, 128 neurons) to capture spatiotemporal complexity.

5.2 Performance Comparison

The solutions for 1D, 2D, and 3D heat equations obtained from the optimal PINN configurations in Table 5.1 resulted in the following performance metrics:

Table 5.2: Comprehensive performance metrics

Dimension	Rank	Method	Relative L_2	Time (s)	Memory (MB)
1D	1	PINN	0.002762	58.16	67.0
	2	Explicit FDM	0.007513	0.003	<1
	3	Crank-Nicolson FDM	0.007697	0.009	<1
	4	Implicit FDM	0.007881	0.016	<1
2D	1	PINN	0.008447	199.24	919.0
	2	Explicit FDM	0.012117	0.005	<1
	3	Crank-Nicolson FDM	0.012398	0.040	<1
	4	Implicit FDM	0.012680	0.039	<1
3D	1	PINN	0.015825	888.28	4503.0
	2	Explicit FDM	0.016458	0.010	<1
	3	Crank-Nicolson FDM	0.016846	2.405	<1
	4	Implicit FDM	0.017233	2.472	<1

Table 5.2 reveals that PINNs achieve lower errors compared to traditional FDM schemes across all dimensions. For 1D, PINN achieves relative L_2 error of 0.002762, outperforming the best FDM method by 63%. In 2D, PINN maintains 30% lower error than explicit FDM. For 3D, PINN achieves 0.015825 relative L_2 error, marginally better than explicit FDM’s 0.016458. However, this accuracy advantage comes at significant computational cost. PINNs require substantially longer training times: 58.16s (1D), 199.24s (2D), and 888.28s (3D) compared to FDM’s sub-second to few-second execution. Memory consumption scales dramatically with dimension, reaching 4503 MB for 3D—over $4000\times$ higher than FDM methods.

The results demonstrate PINN’s dimension-dependent performance characteristics. In lower dimensions (1D, 2D), PINNs provide clear accuracy benefits despite computational overhead. For 3D problems, the marginal accuracy improvement (3.8% over explicit FDM) may not justify the 88,000 $\times$ longer training time and massive memory

requirements for grid-based problems with known analytical solutions. These findings align with observations in [6, 21, 32, 36, 38] regarding PINN complexity in high-dimensional domains.

Moreover, the impact of errors and the agreement of PINN's solution along with the solutions from all the FDM schemes, and the analytical solution (taken as standard of benchmark) measured for selected evaluation times: [1.0, 30.0, 60.0] seconds from the complete collection (4.7) and grid points: (4.8) are shown in Figures 5.1, 5.2, and 5.3. Figure 5.1 is a simple line plot for 1D heat equation. Similarly, the Figure 5.2 is a cross-sectional line plot at $y = 0.5$ for 2D heat equation, and Figure 5.3 is a cross-sectional line plot at $y = 0.5$ and $z = 0.5$ for 3D heat equation.

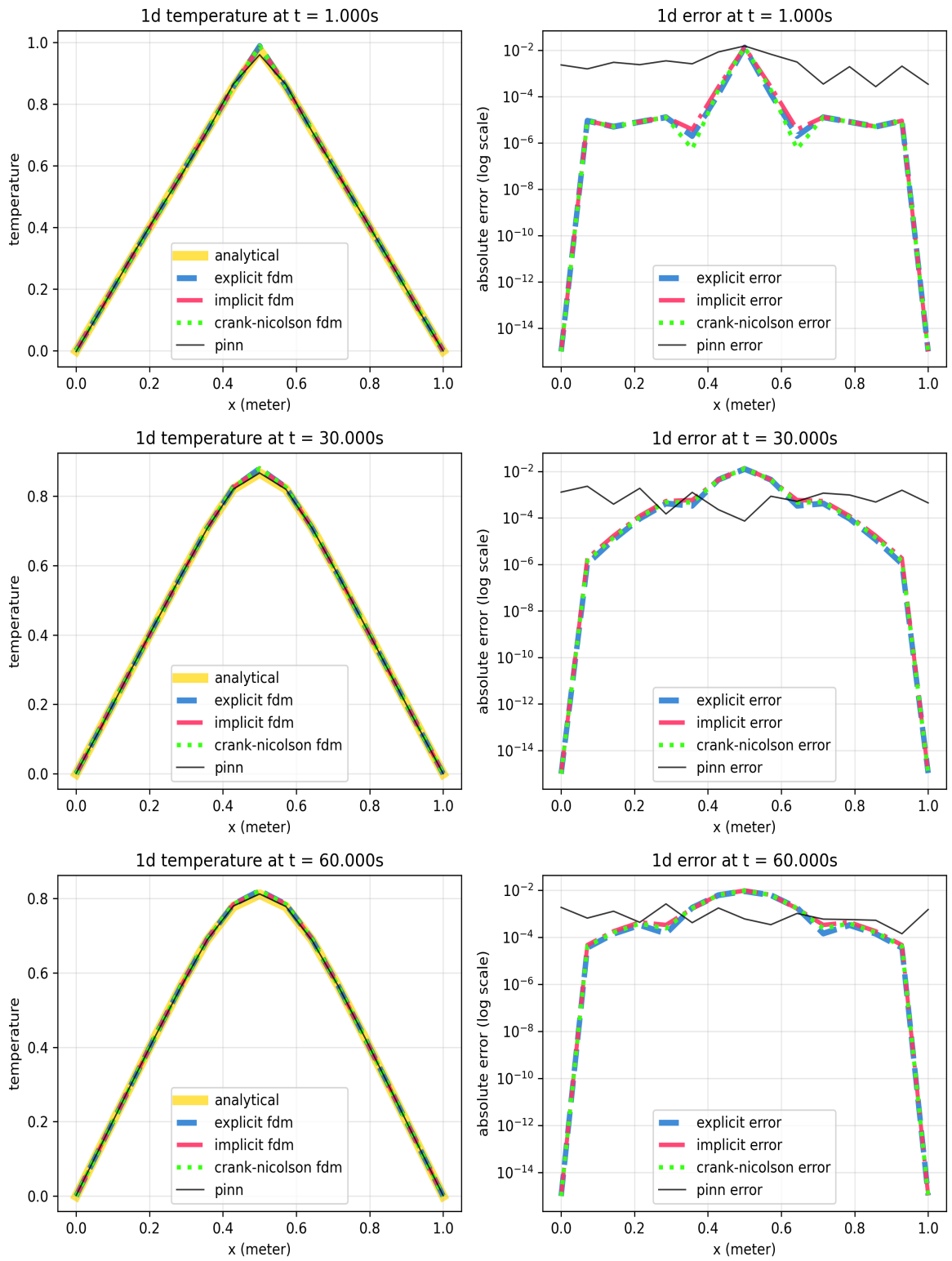


Figure 5.1: 1D heat equation: solution and error comparison.

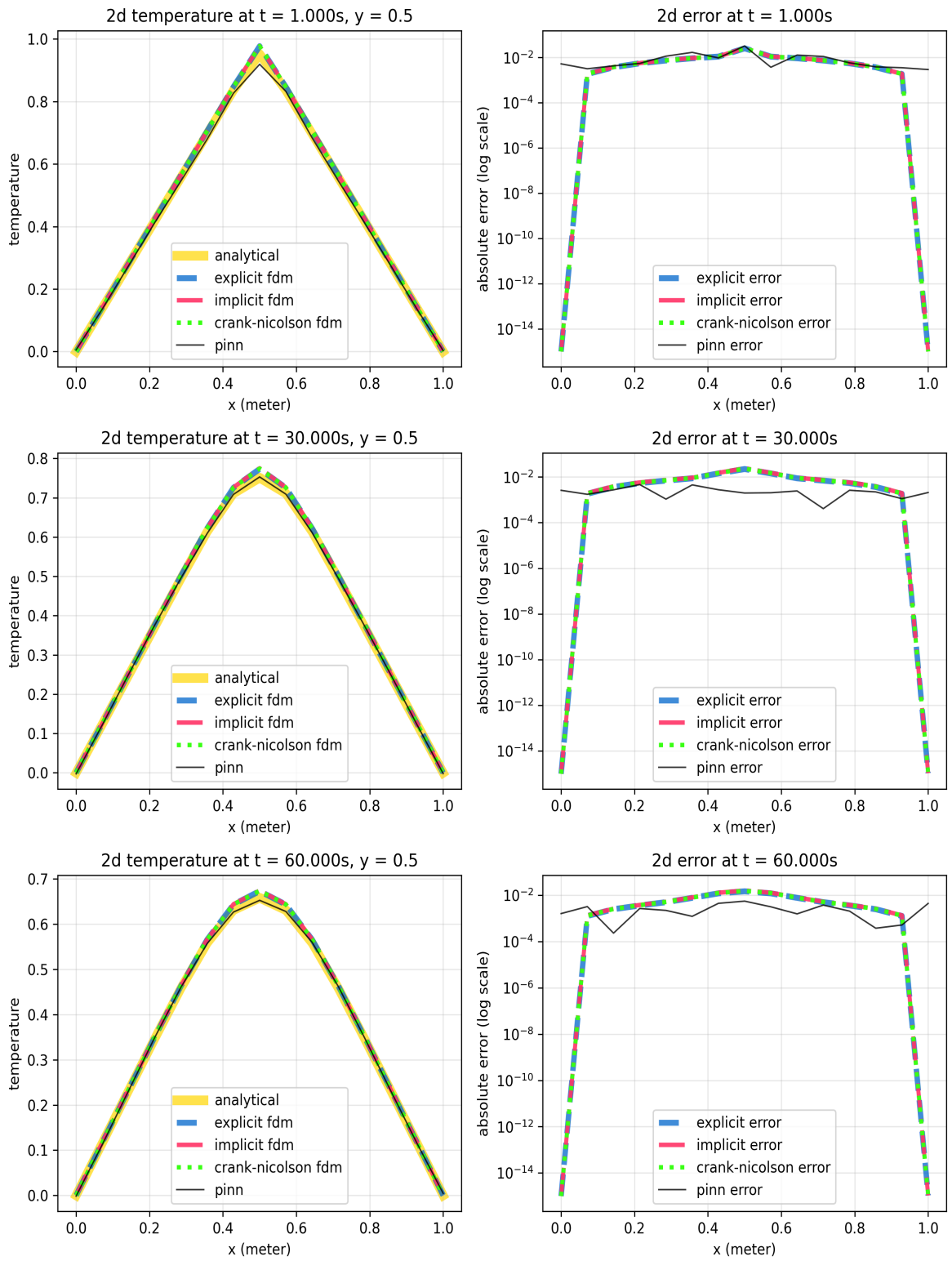


Figure 5.2: 2D heat equation: solution and error comparison.

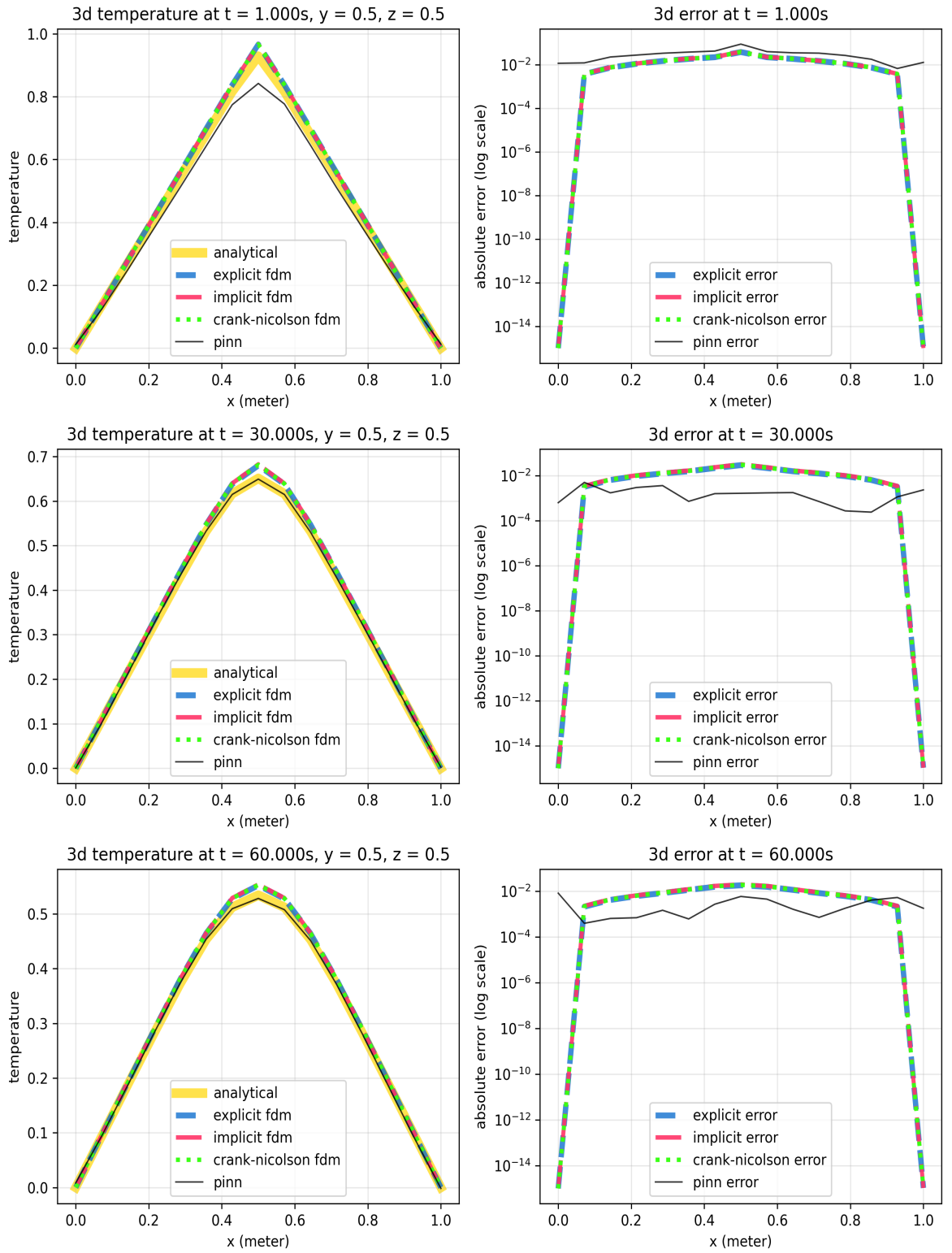


Figure 5.3: 3D heat equation: solution and error comparison.

5.3 Accuracy Analysis

Table 5.3 demonstrates that PINNs achieve superior accuracy compared to traditional FDM methods across all dimensions. For 1D, PINN reaches 99.723837% accuracy, outperforming all FDM schemes (explicit: 99.248670%, implicit: 99.211881%, Crank-Nicolson: 99.230270%). In 2D, PINN maintains 99.155306% accuracy versus FDM’s $\sim 98.7\%$. For 3D, PINN achieves 98.417463%, exceeding explicit FDM (98.354163%), implicit FDM (98.276686%), and Crank-Nicolson (98.315450%).

The accuracy advantage remains consistent across dimensions, with PINNs showing 0.5% improvement in 1D, 0.4% in 2D, and 0.1% in 3D over the best FDM method. This pattern confirms the error trends in Table 5.2, demonstrating PINN’s superior precision for this class of problems despite higher computational cost.

Table 5.3: Accuracy comparison (%)

Dimension	Method	Accuracy
1D	Explicit FDM	99.248670
	Implicit FDM	99.211881
	Crank-Nicolson FDM	99.230270
	PINN	99.723837
2D	Explicit FDM	98.788291
	Implicit FDM	98.731993
	Crank-Nicolson FDM	98.760156
	PINN	99.155306
3D	Explicit FDM	98.354163
	Implicit FDM	98.276686
	Crank-Nicolson FDM	98.315450
	PINN	98.417463

5.4 Discussion

PINNs achieved superior accuracy across all dimensions by leveraging automatic differentiation and adaptive loss weighting. Using LHS-generated collocation points, PINNs approximated solutions without structured grids, avoiding discretization errors inherent to FDM. Adaptive GradNorm weighting dynamically balanced PDE, boundary, and initial condition losses (4.19), (4.21), (4.22), proving particularly effective in 2D and 3D where loss term magnitudes varied significantly during training.

FDM schemes maintained computational efficiency with sub-second to few-second execution times. Explicit schemes were fastest but required CFL stability constraints (4.10). Implicit and Crank-Nicolson schemes offered unconditional stability at the cost of solving linear systems, with execution time increasing in 3D (2.4s for Crank-Nicolson, 2.5s for implicit).

PINNs demonstrated mesh-free flexibility and higher accuracy but required substantially more resources: 58-888 seconds training time and 67-4503 MB memory. The trade-off between PINN accuracy and FDM efficiency suggests dimensional considerations: PINNs excel in low-dimensional problems where accuracy justifies computational cost, while FDM remains practical for high-dimensional cases prioritizing speed. For problems with complex geometries PINN’s mesh-free advantage becomes compelling despite overhead.

5.4.1 Loss Convergence

The loss convergence behavior across 6,000 training epochs varied significantly by dimension. For 1D (Figure 5.4), all loss components exhibited smooth monotonic decay, with PDE loss converging from 10^{-4} to 10^{-8} , boundary condition loss from 10^0 to 10^{-6} , and initial condition loss from 10^{-1} to 10^{-4} . The total loss decreased steadily from 10^0 to 10^{-4} without significant oscillations.

In 2D (Figure 5.5), training dynamics became more complex. While PDE loss maintained smooth convergence from 10^{-4} to 10^{-7} , both boundary and initial condition losses developed periodic oscillations after epoch 2000, with boundary loss fluctuating between 10^{-5} and 10^{-3} and initial condition loss between 10^{-5} and 10^{-2} . Despite these oscillations, the total loss continued decreasing to $\sim 10^{-5}$, and final accuracy remained optimal (99.155%).

For 3D (Figure 5.6), all loss components exhibited persistent high-frequency oscillations throughout training. PDE loss converged smoothly from 10^{-5} to 10^{-7} , but boundary and initial condition losses showed significant variance, with boundary loss oscillating

around 10^{-4} to 10^{-3} and initial condition loss between 10^{-4} and 10^{-2} after initial convergence. The total loss stabilized near 10^{-4} with sustained oscillations, yet achieved competitive accuracy (98.417%).

These oscillation patterns, particularly prominent in 2D and 3D boundary/initial condition losses, reflect the challenge of simultaneously satisfying multiple physics constraints in higher-dimensional spaces. The Adaptive GradNorm weighting strategy successfully maintained convergence despite these dynamics, as evidenced by the superior final accuracies compared to FDM methods.

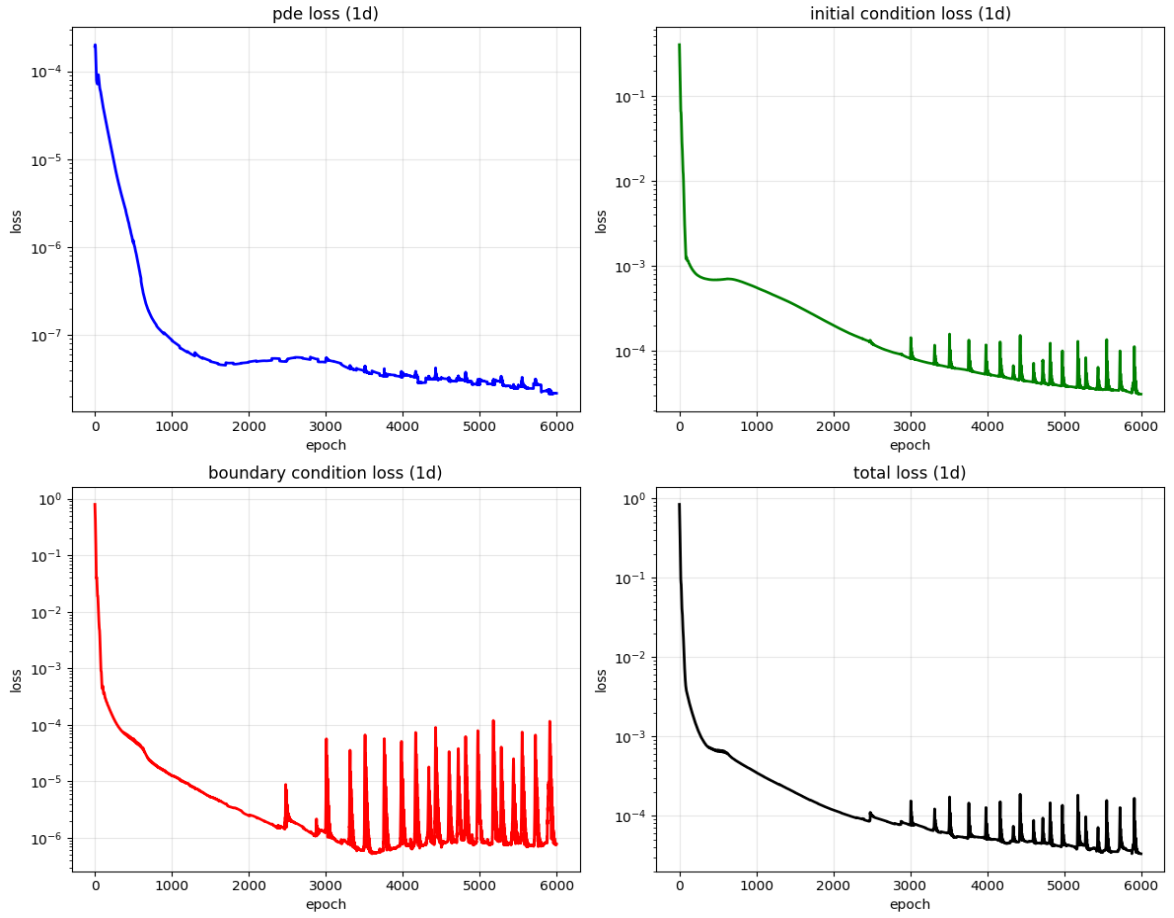


Figure 5.4: Loss convergence of 1D heat equation by PINN

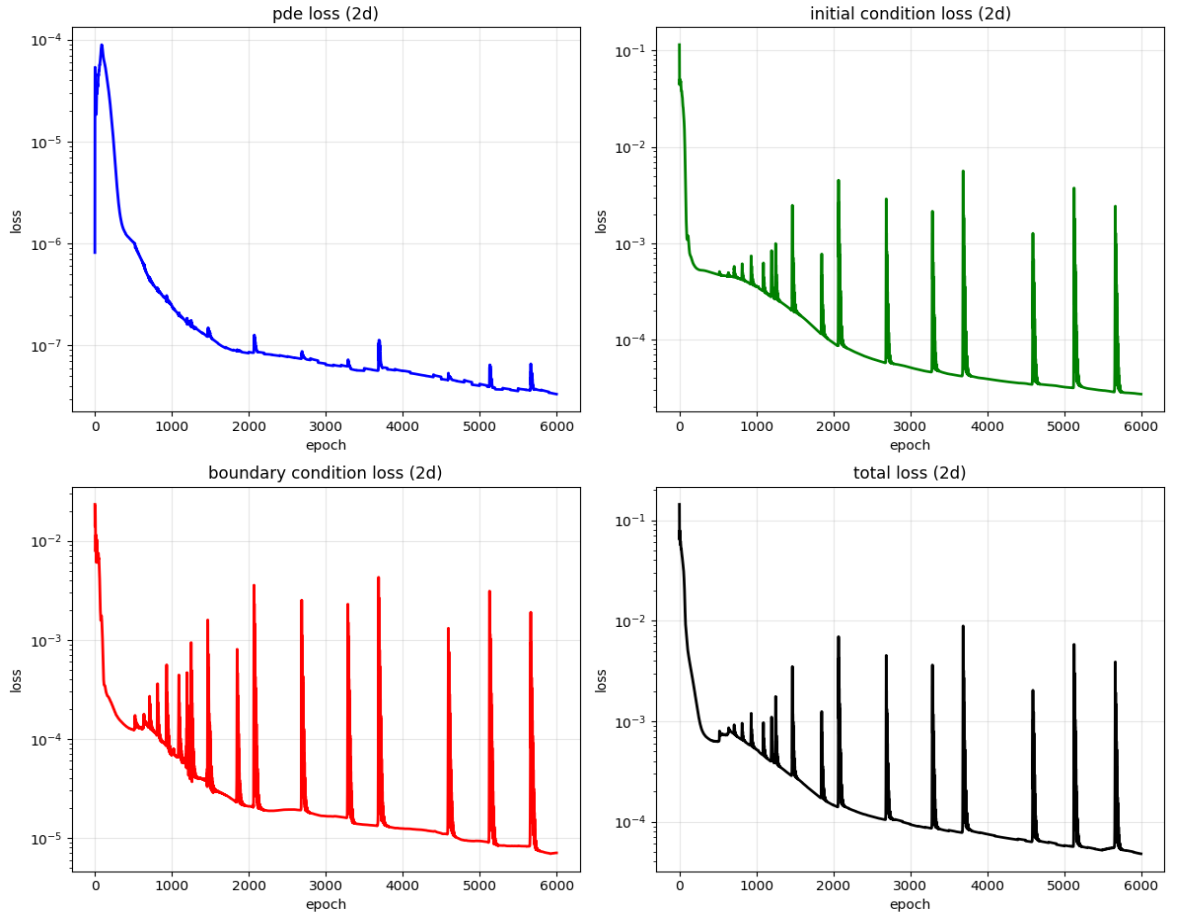


Figure 5.5: Loss convergence of 2D heat equation by PINN

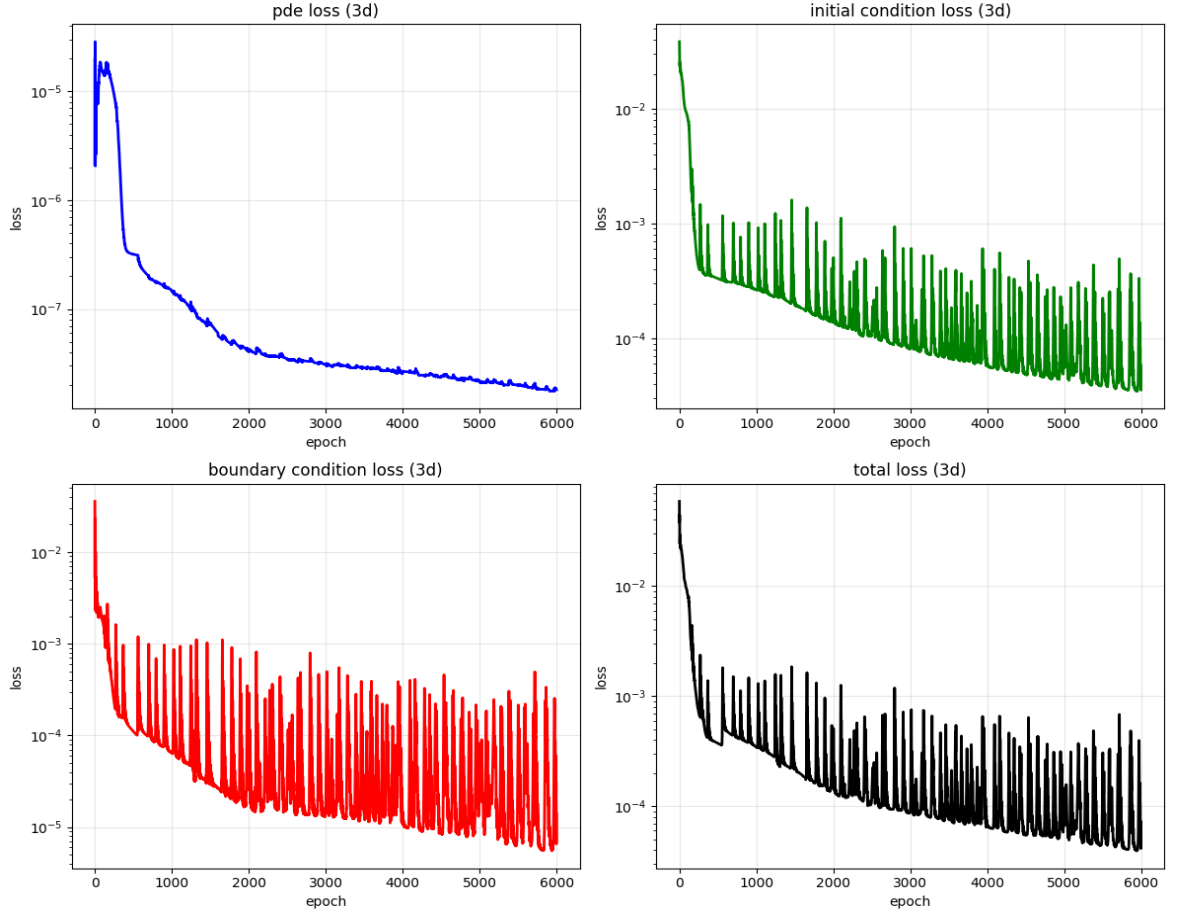


Figure 5.6: Loss convergence of 3D heat equation by PINN

5.4.2 Benchmarks

The computational performance analysis reveals distinct trade-offs between methods:

- **Explicit FDM** achieves fastest execution (<0.01 s avg.) with minimal memory (<1 MB), requiring CFL stability constraints.
- **Implicit Methods** (Implicit, Crank-Nicolson) maintain low memory (<1 MB avg.)
- **PINNs** require significantly more memory (1830 MB avg.) and moderate training time (382s avg.), but demonstrate superior accuracy across all dimensions.

Table 5.4: Average computational performance comparison across all dimensions

Method	Avg. Time (s)	Avg. Memory (MB)
Explicit FDM	0.006	<1
Implicit FDM	0.842	<1
Crank-Nicolson FDM	0.818	<1
PINN	381.89	1829.67

Table 5.5 compares spatiotemporal discretization between FDM grid nodes and PINN collocation samples. The efficiency multiplier (FDM nodes / PINN samples) demonstrates dimensional scaling: PINNs use $4\times$ more samples in 1D and $2.67\times$ more in 2D, but achieve $2.8\times$ fewer samples than FDM in 3D while maintaining superior accuracy.

Table 5.5: Spatiotemporal discretization comparison

Dimension	FDM Grid Nodes	PINN Collocation Samples	Efficiency Multiplier
1D	1,500	6,000	0.25
2D	22,500	60,000	0.375
3D	337,500	120,000	2.8125

These results highlight the fundamental trade-off: FDM methods offer computational efficiency for lower dimensions but require exponentially increasing grid nodes with dimension ($1,500 \rightarrow 22,500 \rightarrow 337,500$), while PINNs maintain linear scaling in collocation samples ($6,000 \rightarrow 60,000 \rightarrow 120,000$) and achieve superior accuracy at the cost of memory overhead. For 3D problems, PINNs demonstrate practical advantages—using $2.8\times$ fewer discretization points while achieving better accuracy than explicit FDM.

5.5 Limitations

FDM accuracy depends on grid resolution, with exponential memory growth in 3D (337,500 nodes). Explicit schemes require CFL stability constraints (4.10), limiting time step sizes. PINNs face training stability challenges and high memory consumption (4503 MB for 3D, Table 5.2). Loss oscillations appear in 2D/3D despite Adaptive GradNorm weighting [37], as shown in Figures 5.5 and 5.6. Collocation point requirements (6,000-120,000) balance accuracy against computational cost [7, 38].

Figure 5.7 demonstrates execution time scaling: PINNs require 58-888s training but maintain linear sample growth, while FDM shows exponential node scaling with dimension. Figure 5.8 demonstrates the efficiency ratio on comparing FDM nodes against PINN samples. Figure 5.9 reveals PINN’s superior accuracy across all dimensions despite computational overhead.

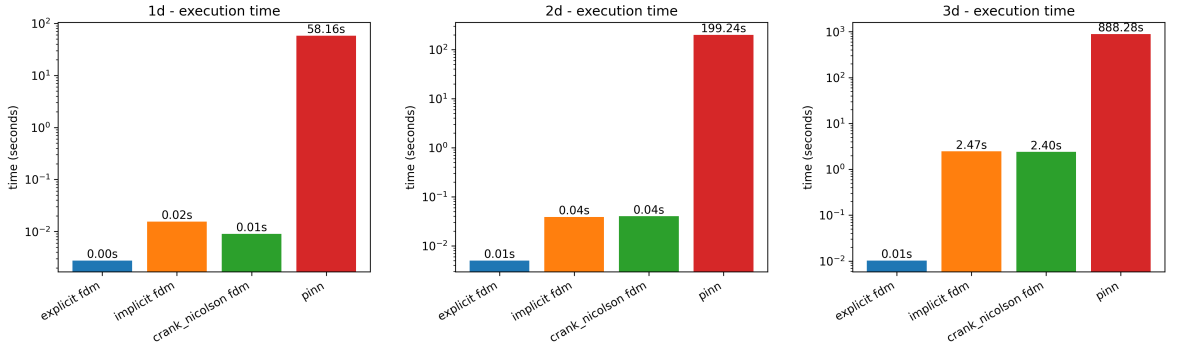


Figure 5.7: Execution time comparison across dimensions for each methods

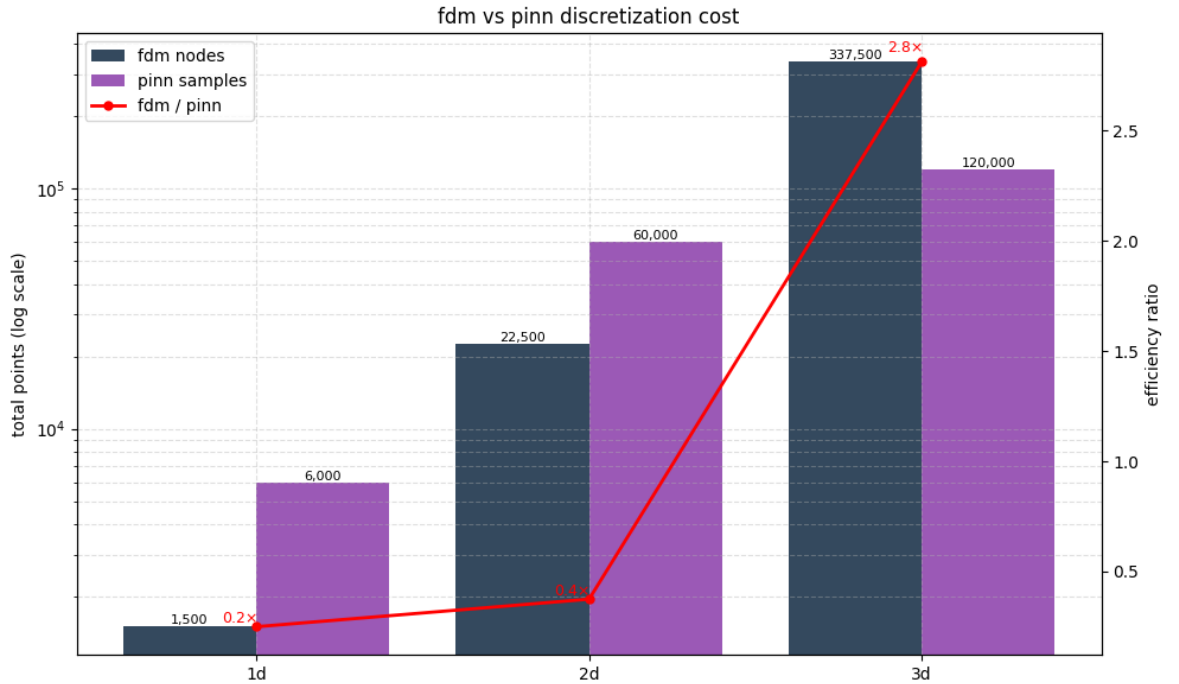


Figure 5.8: FDM vs PINN discretization cost

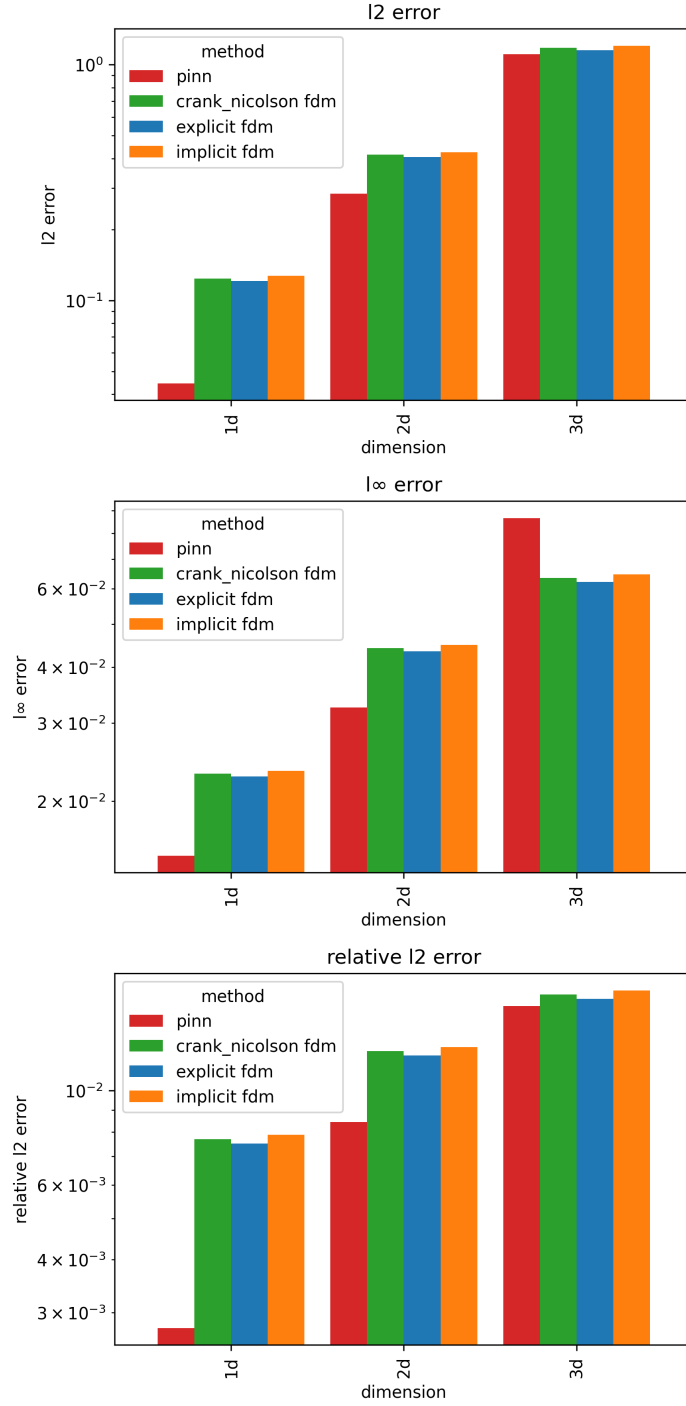


Figure 5.9: Error metrics (L_2 , L_∞ , Relative L_2) by dimension

CHAPTER 6

Conclusion

FDM and PINNs were implemented for 1D, 2D, and 3D heat equations. FDM achieves computational efficiency for structured domains, while PINNs provide mesh-free flexibility with superior accuracy across all dimensions. Here, PINNs outperform FDM in accuracy: 1D (0.28% error vs 0.75%), 2D (0.84% vs 1.21%), and 3D (1.58% vs 1.65%). However, PINNs require significantly more resources: 1D (58s vs 0.003s, 67 MB vs <1 MB), 2D (199s vs 0.005s, 919 MB vs <1 MB), and 3D (888s vs 0.010s for explicit FDM, 4503 MB vs <1 MB). Table 5.5 reveals the scaling advantage: FDM nodes grow exponentially ($1,500 \rightarrow 22,500 \rightarrow 337,500$), while PINN samples scale linearly ($6,000 \rightarrow 60,000 \rightarrow 120,000$). For 3D, PINNs use $2.8\times$ fewer discretization points while achieving better accuracy. Quantitative comparisons from Tables 5.2 and 5.5: 1D explicit FDM is $19,387\times$ faster than PINN; 2D explicit FDM is $39,848\times$ faster than PINN; 3D explicit is $86,205\times$ faster than PINN.

Both methods validated against analytical solutions provide complementary approaches: FDM for efficiency in lower dimensions, PINNs for accuracy and mesh-free advantages in complex geometries.

CHAPTER 7

Future Works

Future enhancements include:

- **Advanced adaptive sampling:** Implement residual-based adaptive sampling to reduce collocation points while preserving accuracy [7, 38].
- **Improved loss weighting:** Develop enhanced schemes to mitigate oscillations observed in Figures 5.5 and 5.6 [37, 42, 46].
- **Frozen noise protocol:** Incorporate stochastic boundary conditions using fixed noise trajectories (Gaussian white noise $\mathcal{N}(0, \sigma^2)$) to model real-world uncertainties while maintaining reproducibility [46].

Bibliography

- [1] Lawrence Craig Evans, *Partial Differential Equations*, Graduate studies in mathematics, American Mathematical Society, 2010.
- [2] Randall J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*, Other Titles in Applied Mathematics, Society for Industrial and Applied Mathematics, 2007.
- [3] Maziar Raissi, Paris Perdikaris, and George Karniadakis, *Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Non-linear Partial Differential Equations*, Journal of Computational Physics **378** (2018), DOI 10.1016/j.jcp.2018.10.045.
- [4] George Karniadakis, Yannis Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang, *Physics-informed machine learning*, Nature Reviews Physics, posted on 2021, 1–19, DOI 10.1038/s42254-021-00314-5.
- [5] Horatio Scott Carslaw and John Conrad Jaeger, *Conduction of Heat in Solids*, Oxford science publications, Clarendon Press, 1986.
- [6] Lu Lu, Xuhui Meng, Zhiping Mao, and George Karniadakis, *DeepXDE: A Deep Learning Library for Solving Differential Equations*, SIAM Review **63** (2021), 208–228, DOI 10.1137/19M1274067.
- [7] Ameya Jagtap, Kenji Kawaguchi, and George Karniadakis, *Adaptive activation functions accelerate convergence in deep and physics-informed neural networks*, Journal of Computational Physics **404** (2019), 109136, DOI 10.1016/j.jcp.2019.109136.
- [8] Walter Alexander Strauss, *Partial Differential Equations: An Introduction*, Wiley, 1992.
- [9] Yunus A Çengel, *Heat Transfer: A Practical Approach with EES CD*, Heat Transfer: A Practical Approach, McGraw-Hill, 2002.
- [10] John Crank, *The Mathematics of Diffusion*, Oxford science publications, Clarendon Press, 1979.
- [11] Frank P. Incropera, *Fundamentals of Heat and Mass Transfer*, Wiley, 2007.
- [12] Bureau International des Poids et Mesures, *Le Système international d’unités/The International System of Units*, 9th, 2024.
- [13] G.D. Smith, *Numerical Solution of Partial Differential Equations: Finite Difference Methods*, Oxford applied mathematics and computing science series, Clarendon Press, 1985.
- [14] M. McKay, Richard Beckman, and William Conover, *A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output From a Computer Code*, Technometrics **21** (1979), 239–245, DOI 10.1080/00401706.1979.10489755.
- [15] Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind, *Automatic Differentiation in Machine Learning: a Survey*, Journal of Machine Learning Research **18** (2018), no. 153, 1–43.

- [16] Gerald Recktenwald, *Finite-Difference Approximations to the Heat Equation*, Mechanical Engineering **10** (2004).
- [17] Benyam Mebrate, *Numerical Solution of a One Dimensional Heat Equation with Dirichlet Boundary Conditions*, American Journal of Applied Mathematics **3** (2015), 305, DOI 10.11648/j.ajam.20150306.20.
- [18] Omar Abdullah Ajeel and Awni M. Gaftan, *Using Crank-Nicolson Numerical Method to solve Heat-Diffusion Problem*, Tikrit Journal of Pure Science **28** (2023), no. 3, 101–104, DOI 10.25130/tjps.v28i3.1434.
- [19] Anirudh Kalyan and Sundararajan Natarajan, *Numerical Simulation of Transient Heat Conduction With Moving Heat Source Using Neural Networks*, International Journal of Mechanical System Dynamics **5** (2025), no. 2, 345–353, DOI 10.1002/msd2.70031.
- [20] Bo Zhang, *Discovering physics-informed neural networks model for solving partial differential equations through evolutionary computation*, Swarm and Evolutionary Computation **88** (2024), DOI 10.1016/j.swevo.2024.101589.
- [21] Weiwei Zhang, Wei Suo, Jiahao Song, and Wenbo Cao, *Physics Informed Neural Networks (PINNs) as intelligent computing technique for solving partial differential equations: Limitation and Future prospects*, 2024.
- [22] Di He, Wenlei Shi, Shanda Li, Xiaotian Gao, Jia Zhang, Jiang Bian, Liwei Wang, and Tie-Yan Liu, *Learning Physics-Informed Neural Networks without Stacked Back-propagation*, 2022.
- [23] Navid Zobeiry and Keith Humfeld, *A Physics-Informed Machine Learning Approach for Solving Heat Transfer Equation in Advanced Manufacturing and Engineering Applications*, Engineering Applications of Artificial Intelligence **101** (2020), DOI 10.1016/j.engappai.2021.104232.
- [24] Shengze Cai, Zhicheng Wang, Sifan Wang, Paris Perdikaris, and George Karniadakis, *Physics-Informed Neural Networks (PINNs) for Heat Transfer Problems*, Journal of Heat Transfer **143** (2021), DOI 10.1115/1.4050542.
- [25] Samman Aryal, *Comparison of 2D heat equation solutions using PINNs and FEM*, 2024.
- [26] Saakaar Bhatnagar, Andrew Comerford, and Araz Banaeizadeh, *Physics Informed Neural Networks for Modeling of 3D Flow-Thermal Problems with Sparse Domain Data*, Journal of Machine Learning for Modeling and Computing **5** (2024), DOI 10.1615/JMachLearnModelComput.2024051540.
- [27] Zebin Xing, Heng Cheng, and Jing Cheng, *Deep Learning Method Based on Physics-Informed Neural Network for 3D Anisotropic Steady-State Heat Conduction Problems*, Mathematics **11** (2023), 4049, DOI 10.3390/math11194049.
- [28] Jenny Farmer, Chad A. Oian, Brett A. Bowman, and Taufiqar Khan, *Empirical loss weight optimization for PINN modeling laser bio-effects on human skin for the 1D heat equation*, Machine Learning with Applications **16** (2024), 100563, DOI 10.1016/j.mlwa.2024.100563.
- [29] Maziar Raissi, Paris Perdikaris, Nazanin Ahmadi Daryakenari, and George Karniadakis, *Physics-Informed Neural Networks and Extensions*, 2024.
- [30] Gianluca Fabiani, Erik Bollt, Constantinos Siettos, and Athanasios Yannacopoulos, *Stability Analysis of Physics-Informed Neural Networks for Stiff Linear Differential Equations*, 2024.
- [31] Scott Rolfo, *Machine Learning-driven Numerical Solutions to Partial Differential Equations*, J Appl Computat Math **13** (2024), 571.

- [32] Aditi Krishnapriyan, Asghar Gholami, Shandian Zhe, Robert Kirby, and Michael Mahoney, *Characterizing possible failure modes in physics-informed neural networks*, 2021.
- [33] Keith William Morton and D.F. Mayers, *Numerical Solution of Partial Differential Equations: An Introduction*, Cambridge University Press, 2005.
- [34] Richard Courant, K. Friedrichs, and H. Lewy, *On the partial difference equations of mathematical physics*, IBM J. Res. Dev. **11** (1967), no. 2, 215–234, DOI 10.1147/rd.112.0215.
- [35] Xavier Glorot and Y. Bengio, *Understanding the difficulty of training deep feedforward neural networks*, Journal of Machine Learning Research - Proceedings Track **9** (2010), 249–256.
- [36] Jorge F. Urbán, Petros Stefanou, and José A. Pons, *Unveiling the optimization process of physics informed neural networks: How accurate and competitive can PINNs be?*, Journal of Computational Physics **523** (2025), 113656, DOI 10.1016/j.jcp.2024.113656.
- [37] Sifan Wang, Yujun Teng, and Paris Perdikaris, *Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks*, SIAM Journal on Scientific Computing **43** (2021), A3055–A3081, DOI 10.1137/20M1318043.
- [38] Sifan Wang, Xinling Yu, and Paris Perdikaris, *When and why PINNs fail to train: A Neural Tangent Kernel perspective*, Journal of Computational Physics, posted on 2021, 110768, DOI 10.1016/j.jcp.2021.110768.
- [39] Brett Bowman, Chad Oian, Jason Kurz, Taufiqar Khan, Eddie Gil, and Nick Gamez, *Physics-Informed Neural Networks for the Heat Equation with Source Term under Various Boundary Conditions*, Algorithms **16** (2023), 428, DOI 10.3390/a16090428.
- [40] Maciej Sikora, Patryk Krukowski, Anna Paszyńska, and Maciej Paszyński, *Comparison of Physics Informed Neural Networks and Finite Element Method Solvers for advection-dominated diffusion problems*, Journal of Computational Science **81** (2024), 102340, DOI 10.1016/j.jocs.2024.102340.
- [41] Svetislav Savović, Miloš Ivanović, and Rui Min, *A Comparative Study of the Explicit Finite Difference Method and Physics-Informed Neural Networks for Solving the Burgers’ Equation*, Axioms **12** (2023), no. 10, 982, DOI 10.3390/axioms12100982.
- [42] Zixue Xiang, Wei Peng, Wen Yao, and Weien Zhou, *Hybrid Finite-Difference Physics-Informed Neural Networks Partial Differential Equation Solver for Complex Geometries*, Journal of Thermophysics and Heat Transfer, posted on 2025, 1–18, DOI 10.2514/1.T7077.
- [43] Atakan Aygun and Ali Karakus, *Physics Informed Neural Networks for Two Dimensional Incompressible Thermal Convection Problems*, Journal of Thermal Sciences and Technology, posted on 2022, 221–232, DOI 10.47480/isibted.1194992.
- [44] Hansaem Oh and Gwanghyun Jo, *Physics-informed neural network for the heat equation under imperfect contact conditions and its error analysis*, AIMS Mathematics **10** (2025), 7920–7940, DOI 10.3934/math.2025364.
- [45] Tamara Grossmann, Urszula Komorowska, Jonas Latz, and Carola-Bibiane Schönlieb, *Can physics-informed neural networks beat the finite element method?*, IMA Journal of Applied Mathematics **89** (2024), DOI 10.1093/imamat/hxae011.
- [46] Rafael Bischof and Michael A. Kraus, *Multi-Objective Loss Balancing for Physics-Informed Deep Learning*, Computer Methods in Applied Mechanics and Engineering **439** (2025), 117914, DOI 10.1016/j.cma.2025.117914.

- [47] Vitor Bueno, Cristiane Faria, Karla Figueiredo, and Fabio dos Santos, *Convergence analysis of physics-informed neural networks and comparison with finite difference methods of the two-dimensional heat equation*, Proceeding Series of the Brazilian Society of Computational and Applied Mathematics **11** (2025), no. 1, 1–7.
- [48] John C. Strikwerda, *Finite Difference Schemes and Partial Differential Equations, Second Edition*, Society for Industrial and Applied Mathematics, 2004.
- [49] R.H. and Tannehill Pletcher J.C. and Anderson, *Computational Fluid Mechanics and Heat Transfer, Second Edition*, Series in Computational and Physical Processes in Mechanics and Thermal Sciences, Taylor & Francis, 1997.